```
// ============================================================================
// 文件：entry/src/main/ets/common/Cms.ets     （共 661 行）
// ============================================================================
import hilog from '@ohos.hilog';
import { Http } from './Http';
import { Site, Category, Vod, VodPage, Line, Episode, JResp, JVod, JClass,
  FilterGroup, FilterValue } from './Model';
/**
 * 内容接口客户端：面向标准 CMS 开放数据接口。
 *
 * 支持三类数据源协议：
 *   type 0 —— XML 数据接口
 *   type 1 —— JSON 数据接口
 *   type 4 —— 返回体多包一层 data 的 JSON 接口（同一套参数）
 * 脚本型实现依赖 JVM 与字节码加载器，本平台不支持，由上层标记为不可用。
 */
/** 一条接口响应缓存。key 即完整 URL（含接口地址与查询串），天然按来源隔离 */
class CmHit {
  key: string = '';
  body: string = '';
  at: number = 0;
}
export class Cms {
  /**
   * 接口响应缓存。
   *
   * 一次列表请求往往 1~3 秒，而用户在分类标签上来回点的概率很高
   * （看看「电影」有没有新片、再回「电视剧」继续）。之前每次切换都重新请求，
   * 回来一次就白等一次。这里加一层按 URL 的短 TTL 缓存：
   *   分类表 5 分钟（基本不变）
   *   列表   90 秒（够「来回切」，又不会把刚更新的片单压太久）
   *   详情   10 分钟（详情几乎不变，但剧集会更新，所以不给太长）
   * 搜索不缓存：关键词千变万化，缓存命中率低，还容易把「刚试过的搜索」结果留旧。
   */
  private static hits: CmHit[] = [];
  private static readonly HIT_MAX: number = 30;
  private static readonly HIT_CHARS: number = 3000000;
  private static readonly TTL_CAT: number = 300000;
  private static readonly TTL_LIST: number = 90000;
  private static readonly TTL_DETAIL: number = 600000;
  /** 读缓存；过期或没有都返回空串 */
  private static hit(url: string, ttl: number): string {
    const now: number = Date.now();
    for (let i = 0; i < Cms.hits.length; i++) {
      if (Cms.hits[i].key === url) {
        if (now - Cms.hits[i].at <= ttl) {
          return Cms.hits[i].body;
        }
        Cms.hits.splice(i, 1);   // 过期就地清掉，避免数组越攒越长
        return '';
```

```
      }
    }
    return '';
  }
  /** 写缓存：同 URL 覆盖、超条数按最旧淘汰、超总字数从最旧开始丢 */
  private static keep(url: string, body: string): void {
    if (body.length === 0) {
      return;                   // 空响应不缓存，否则一次失败会「粘」住 90 秒
    }
    const e = new CmHit();
    e.key = url;
    e.body = body;
    e.at = Date.now();
    for (let i = 0; i < Cms.hits.length; i++) {
      if (Cms.hits[i].key === url) {
        Cms.hits[i] = e;
        return;
      }
    }
    Cms.hits.push(e);
    while (Cms.hits.length > Cms.HIT_MAX) {
      Cms.hits.shift();
    }
    let total: number = 0;
    for (let i = 0; i < Cms.hits.length; i++) {
      total += Cms.hits[i].body.length;
    }
    while (total > Cms.HIT_CHARS && Cms.hits.length > 1) {
      total -= Cms.hits[0].body.length;
      Cms.hits.shift();
    }
  }
  /**
   * 缓存透明的一层：命中直接返回，未命中请求后落缓存。
   * 注意不能叫 text —— 类里已有一个 `Cms.text(s)` 负责剥 HTML 标签。
   */
  private static async fetch(url: string, ttl: number, ms: number): Promise<string
> {
    const has: string = Cms.hit(url, ttl);
    if (has.length > 0) {
      hilog.info(0x1000, 'BOXN', 'cache HIT  %{public}s', Cms.short(url));
      return has;
    }
    const t0: number = Date.now();
    const txt: string = await Http.text(url, undefined, ms);
    hilog.info(0x1000, 'BOXN', 'cache MISS %{public}s  %{public}d ms',
      Cms.short(url), Date.now() - t0);
    Cms.keep(url, txt);
    return txt;
  }
```

```
  /** 日志里只留参数部分，URL 太长会把 hilog 刷屏 */
  private static short(url: string): string {
    const i: number = url.indexOf('?');
    const q: string = i >= 0 ? url.substring(i + 1) : url;
    return q.length > 48 ? q.substring(0, 48) : q;
  }
  /** 用户更换配置后主动清空，避免旧响应还留着 */
  static clearCache(): void {
    Cms.hits = [];
  }
  private site: Site;
  /** 筛选条件尾巴，形如 &area=大陆&year=2025。由点播页切换筛选时塞进来 */
  private extra: string = '';
  constructor(site: Site) {
    this.site = site;
  }
  setExtra(q: string): void {
    this.extra = q;
  }
  // ---------- 公共入口 ----------
  async categories(): Promise<Category[]> {
    const txt: string = await Cms.fetch(this.u('ac=list'), Cms.TTL_CAT, 15000);
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      const cats: Category[] = Cms.catFromXml(txt);
      if (cats.length > 0) {
        const all = new Category();
        all.id = '';
        all.name = '全部';
        cats.unshift(all);
        return cats;
      }
      // 该站的 ?ac=list 直接返回影片列表，只能从条目里反推分类；
      // 这时补一个「全部」(tid 传空串接口会返回全集)，否则会被锁死在单个分类里
      const derived: Category[] = Cms.catsFromVideos(txt);
      if (derived.length > 0) {
        const all = new Category();
        all.id = '';
        all.name = '全部';
        derived.unshift(all);
      }
      return derived;
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    const out: Category[] = [];
    if (root.catList !== undefined) {
      for (let i = 0; i < root.catList.length; i++) {
        const c = new Category();
        c.id = String(root.catList[i].type_id === undefined ? '' : root.catList[i]
.type_id);
        c.name = Cms.s(root.catList[i].type_name);
```

```
        // type_pid 为 0 / 缺省都算顶级分类
        const pidAny: number | undefined = root.catList[i].type_pid;
        const pidRaw: number = pidAny === undefined ? 0 : pidAny;
        c.pid = pidRaw > 0 ? String(pidRaw) : '';
        if (c.id.length > 0 && c.name.length > 0) {
          out.push(c);
        }
      }
    }
    if (out.length === 0 && root.list !== undefined) {
      const seen: string[] = [];
      for (let i = 0; i < root.list.length; i++) {
        const tid: string = String(root.list[i].tid === undefined ? '' : root.list
[i].tid);
        const tn: string = Cms.s(root.list[i].type_name);
        if (tid.length > 0 && tn.length > 0 && seen.indexOf(tid) < 0) {
          seen.push(tid);
          const c = new Category();
          c.id = tid;
          c.name = tn;
          out.push(c);
        }
      }
    }
    if (out.length > 0) {
      // t 传空串多数 CMS 会返回全集，作为默认入口体验更好：
      // 部分站点的分类 id 里只有零星几条，不开「全部」会让人以为没内容
      const all = new Category();
      all.id = '';
      all.name = '全部';
      out.unshift(all);
    }
    return out;
  }
  async list(tid: string, page: number): Promise<VodPage> {
    const q: string = 'ac=videolist&t=' + encodeURIComponent(tid) + '&pg=' + page
+ this.extra;
    const txt: string = await Cms.fetch(this.u(q), Cms.TTL_LIST, 18000);
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      return Cms.listFromXml(txt, page, this.site);
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    const pg = new VodPage();
    pg.page = root.page === undefined ? page : root.page;
    pg.pageCount = root.pagecount === undefined ? 1 : root.pagecount;
    pg.total = root.total === undefined ? 0 : root.total;
    if (root.list !== undefined) {
      for (let i = 0; i < root.list.length; i++) {
        pg.items.push(this.toVod(root.list[i]));
      }
```

```
  }
  return pg;
}
/**
 * 把一个父分类和它的子分类合在一起取第一页。
 *
 * 为什么需要：父分类（如 `电视剧`）通常**没有直属影片**，
 * 直接请求 `t=1` 返回空列表，界面就会显示「这个分类下没有内容」——
 * 但下面明明排着 欧美剧/韩剧/日剧。用户会以为软件坏了。
 * 这里把父 + 子同时请求、按 vod id 去重合并，父分类点进去也有内容。
 *
 * 只取第 1 页且不做翻页：聚合结果本来就是「各类最新的混排」，
 * 继续翻页的语义说不清（翻谁的页？），不如把翻页留给具体的子分类。
 */
async listMerged(ids: string[], page: number): Promise<VodPage> {
  const out = new VodPage();
  out.page = 1;
  out.pageCount = 1;
  const seen: string[] = [];
  const tasks: Promise<VodPage>[] = [];
  for (let i = 0; i < ids.length; i++) {
    tasks.push(this.list(ids[i], page));
  }
  const got: VodPage[] = await Promise.all(tasks);
  for (let i = 0; i < got.length; i++) {
    const items: Vod[] = got[i].items;
    for (let k = 0; k < items.length; k++) {
      if (seen.indexOf(items[k].id) < 0) {
        seen.push(items[k].id);
        out.items.push(items[k]);
      }
    }
  }
  out.total = out.items.length;
  return out;
}
async detail(id: string): Promise<Vod> {
  const q: string = 'ac=detail&ids=' + encodeURIComponent(id);
  const txt: string = await Cms.fetch(this.u(q), Cms.TTL_DETAIL, 18000);
  if (this.site.type === 0 || Cms.looksXml(txt)) {
    return Cms.detailFromXml(txt, this.site, id);
  }
  const root: JResp = Cms.root(Cms.parse(txt));
  if (root.list !== undefined && root.list.length > 0) {
    return this.toVod(root.list[0]);
  }
  throw new Error('接口没有返回详情数据');
}
/** 搜索；不同站对 ac / wd 的接受度不同，做两级兜底 */
async search(kw: string): Promise<Vod[]> {
```

```
    const e: string = encodeURIComponent(kw);
    let items: Vod[] = [];
    try {
      const txt: string = await Http.text(this.u('ac=videolist&wd=' + e), undefine
d, 12000);
      items = this.pickList(txt);
    } catch (err) {
      items = [];
    }
    if (items.length === 0) {
      const txt2: string = await Http.text(this.u('wd=' + e), undefined, 12000);
      items = this.pickList(txt2);
    }
    return items;
  }
  private pickList(txt: string): Vod[] {
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      return Cms.listFromXml(txt, 1, this.site).items;
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    const out: Vod[] = [];
    if (root.list !== undefined) {
      for (let i = 0; i < root.list.length; i++) {
        out.push(this.toVod(root.list[i]));
      }
    }
    return out;
  }
  // ---------- 内部 ----------
  private u(params: string): string {
    const a: string = this.site.api;
    if (a.endsWith('?')) {
      return a + params;
    }
    if (a.indexOf('?') >= 0) {
      return a + '&' + params;
    }
    return a + '?' + params;
  }
  private toVod(j: JVod): Vod {
    const v = new Vod();
    v.id = String(j.vod_id === undefined ? '' : j.vod_id);
    v.name = Cms.s(j.vod_name);
    v.pic = Cms.s(j.vod_pic);
    v.remark = Cms.s(j.vod_remarks);
    v.year = Cms.s(j.vod_year);
    v.area = Cms.s(j.vod_area);
    v.actor = Cms.s(j.vod_actor);
    v.director = Cms.s(j.vod_director);
    v.content = Cms.plain(Cms.s(j.vod_content).length > 0 ? Cms.s(j.vod_content) :
```

```
Cms.s(j.vod_blurb));
    v.typeName = Cms.s(j.type_name);
    v.from = Cms.s(j.vod_play_from);
    v.playUrl = Cms.s(j.vod_play_url);
    v.siteKey = this.site.key;
    v.siteName = this.site.name;
    return v;
  }
  // ---------- 静态工具 ----------
  static s(v: string | undefined | null): string {
    if (v === undefined || v === null) {
      return '';
    }
    return String(v).trim();
  }
  static looksXml(txt: string): boolean {
    const t: string = txt.trimStart();
    return t.startsWith('<');
  }
  /** 配置 JSON 容错：去 BOM、去 // 注释行、把保留字 class 改名 */
  static cleanJson(txt: string): string {
    let t: string = txt;
    if (t.charCodeAt(0) === 0xFEFF) {
      t = t.substring(1);
    }
    const raw: string[] = t.split('\n');
    const keep: string[] = [];
    for (let i = 0; i < raw.length; i++) {
      const line: string = raw[i].replace('\r', '');
      if (line.trimStart().startsWith('//')) {
        continue;
      }
      keep.push(line);
    }
    return keep.join('\n').replace(/"class"\s*:/g, '"catList":');
  }
  static parse(txt: string): JResp {
    const t: string = Cms.cleanJson(txt);
    if (t.trim().length === 0) {
      throw new Error('接口返回空内容');
    }
    try {
      return JSON.parse(t) as JResp;
    } catch (err) {
      throw new Error('接口返回的不是合法 JSON（数据源可能已失效）');
    }
  }
  static root(j: JResp): JResp {
    if (j.data !== undefined && j.data !== null) {
      return j.data;
```

```
  }
  return j;
}
/** HTML 实体 + 标签清理，用于简介 */
static plain(s: string): string {
  let r: string = s.replace(/<br\s*\/?>/gi, '\n');
  r = r.replace(/<\/p>/gi, '\n');
  r = r.replace(/<[^>]+>/g, '');
  r = r.replace(/&lt;/g, '<').replace(/&gt;/g, '>').replace(/&quot;/g, '"')
    .replace(/&#39;/g, "'").replace(/ /g, ' ').replace(/&amp;/g, '&');
  r = r.replace(/\n{3,}/g, '\n\n');
  return r.trim();
}
/** XML 节点取值：先取 CDATA 内容，再清标签与实体。
 *  注意顺序——不能先跑 plain()，否则 <![CDATA[片名]]> 会被当成标签整段删掉。 */
static node(block: string, tag: string): string {
  const re = new RegExp('<' + tag + '>([\\s\\S]*?)</' + tag + '>', 'i');
  const m = block.match(re);
  if (m === null) {
    return '';
  }
  return Cms.text(m[1]);
}
static text(s: string): string {
  let r: string = s;
  const c: number = r.indexOf('<![CDATA[');
  if (c >= 0) {
    const e: number = r.indexOf(']]>', c);
    if (e > 0) {
      r = r.substring(c + 9, e);
    }
  }
  return Cms.plain(r);
}
/** 部分站点的 ?ac=list 不返回 <class>，而是直接返回影片列表。
 *  这时从每条影片的 tid/type 反推分类，比"没有分类"更好用。 */
static catsFromVideos(xml: string): Category[] {
  const out: Category[] = [];
  const seen: string[] = [];
  const blocks = xml.match(/<video>[\s\S]*?<\/video>/gi);
  if (blocks === null) {
    return out;
  }
  for (let i = 0; i < blocks.length; i++) {
    const tid: string = Cms.node(blocks[i], 'tid');
    const tn: string = Cms.node(blocks[i], 'type');
    if (tid.length > 0 && tn.length > 0 && seen.indexOf(tid) < 0) {
      seen.push(tid);
      const c = new Category();
      c.id = tid;
```

```
      c.name = tn;
      out.push(c);
    }
  }
  return out;
}
static catFromXml(xml: string): Category[] {
  const out: Category[] = [];
  const re = /<ty\b[^>]*\bid="(\d+)"[^>]*>([\s\S]*?)<\/ty>/gi;
  let m = re.exec(xml);
  while (m !== null) {
    const c = new Category();
    c.id = m[1];
    c.name = Cms.plain(m[2]);
    if (c.name.length > 0) {
      out.push(c);
    }
    m = re.exec(xml);
  }
  return out;
}
static listFromXml(xml: string, page: number, site: Site): VodPage {
  const pg = new VodPage();
  pg.page = page;
  const head = xml.match(/<list\b[^>]*>/i);
  if (head !== null) {
    const p = head[0].match(/\bpage="(\d+)"/i);
    const pc = head[0].match(/\bpagecount="(\d+)"/i);
    const rc = head[0].match(/\brecordcount="(\d+)"/i);
    if (p !== null) {
      pg.page = parseInt(p[1], 10);
    }
    if (pc !== null) {
      pg.pageCount = parseInt(pc[1], 10);
    }
    if (rc !== null) {
      pg.total = parseInt(rc[1], 10);
    }
  }
  const blocks = xml.match(/<video>[\s\S]*?<\/video>/gi);
  if (blocks !== null) {
    for (let i = 0; i < blocks.length; i++) {
      const b: string = blocks[i];
      const v = new Vod();
      v.id = Cms.node(b, 'id');
      v.name = Cms.node(b, 'name');
      v.pic = Cms.node(b, 'pic');
      const note: string = Cms.node(b, 'note');
      v.remark = note.length > 0 ? note : Cms.node(b, 'dt');
      v.typeName = Cms.node(b, 'type');
```

```
      v.siteKey = site.key;
      v.siteName = site.name;
      if (v.name.length > 0) {
        pg.items.push(v);
      }
    }
  }
  return pg;
}
static detailFromXml(xml: string, site: Site, id: string): Vod {
  const blocks = xml.match(/<video>[\s\S]*?<\/video>/gi);
  if (blocks === null || blocks.length === 0) {
    throw new Error('接口没有返回详情数据');
  }
  const b: string = blocks[0];
  const v = new Vod();
  v.id = Cms.node(b, 'id');
  if (v.id.length === 0) {
    v.id = id;
  }
  v.name = Cms.node(b, 'name');
  v.pic = Cms.node(b, 'pic');
  v.remark = Cms.node(b, 'note');
  v.typeName = Cms.node(b, 'type');
  v.year = Cms.node(b, 'year');
  v.area = Cms.node(b, 'area');
  v.actor = Cms.node(b, 'actor');
  v.director = Cms.node(b, 'director');
  v.content = Cms.node(b, 'des');
  v.siteKey = site.key;
  v.siteName = site.name;
  // <dl><dd flag="ckm3u8">第1集$http://...</dd></dl>
  const flags: string[] = [];
  const urls: string[] = [];
  const ddRe = /<dd\b[^>]*\bflag="([^"]*)"[^>]*>([\s\S]*?)<\/dd>/gi;
  let m = ddRe.exec(b);
  while (m !== null) {
    flags.push(m[1]);
    urls.push(Cms.text(m[2]));
    m = ddRe.exec(b);
  }
  if (flags.length === 0) {
    const ddRe2 = /<dd\b[^>]*>([\s\S]*?)<\/dd>/gi;
    let m2 = ddRe2.exec(b);
    let n: number = 1;
    while (m2 !== null) {
      flags.push('线路' + n);
      urls.push(Cms.text(m2[1]));
      n++;
      m2 = ddRe2.exec(b);
```

```
      }
    }
    v.from = flags.join('$$$');
    v.playUrl = urls.join('$$$');
    return v;
  }
  /** vod_play_url / vod_play_from → 线路与剧集 */
  static splitLines(from: string, playUrl: string): Line[] {
    const out: Line[] = [];
    if (playUrl.length === 0) {
      return out;
    }
    const fArr: string[] = from.length > 0 ? from.split('$$$') : [];
    const uArr: string[] = playUrl.split('$$$');
    for (let i = 0; i < uArr.length; i++) {
      const ln = new Line();
      ln.name = i < fArr.length && fArr[i].length > 0 ? fArr[i] : ('线路 ' + (i +
1));
      const segs: string[] = uArr[i].split('#');
      for (let k = 0; k < segs.length; k++) {
        const seg: string = segs[k].trim();
        if (seg.length === 0) {
          continue;
        }
        const ep = new Episode();
        const p: number = seg.indexOf('$');
        if (p > 0) {
          ep.name = seg.substring(0, p).trim();
          ep.url = seg.substring(p + 1).trim();
        } else {
          ep.name = '第 ' + (k + 1) + ' 集';
          ep.url = seg;
        }
        if (ep.url.length > 0 && ep.url.indexOf('://') > 0) {
          ln.eps.push(ep);
        }
      }
      // 标记这条线路能否直接喂给播放器：常见站点第一组是网页地址
      //（如 https://cdn/play/xxx），第二组才是真流（.../index.m3u8）
      let directCount: number = 0;
      for (let k = 0; k < ln.eps.length; k++) {
        if (Cms.isDirect(ln.eps[k].url)) {
          directCount++;
        }
      }
      ln.direct = ln.eps.length > 0 && directCount * 2 >= ln.eps.length;
      if (ln.eps.length > 0) {
        out.push(ln);
      }
    }
```

```
    return out;
  }
  static isDirect(url: string): boolean {
    const u: string = url.toLowerCase();
    return u.indexOf('.m3u8') >= 0 || u.indexOf('.mp4') >= 0 || u.indexOf('.flv')
>= 0 ||
      u.indexOf('.mkv') >= 0 || u.indexOf('.ts') >= 0 || u.indexOf('.m4v') >= 0;
  }
  /**
   * 从站点 ext 里取出该分类的筛选组。
   * ext 形如 {"class":[...],"filters":{"1":[{"key":"area","name":"地区","value":[
{"n":"全部","v":""}]}]}}
   * 没有 ext、不是 JSON、或该分类没配筛选时返回空数组。
   */
  static filtersOf(ext: string, tid: string): FilterGroup[] {
    const out: FilterGroup[] = [];
    const txt: string = ext.trim();
    if (txt.length === 0 || txt.charAt(0) !== '{') {
      return out;
    }
    let j: JExt;
    try {
      j = JSON.parse(txt) as JExt;
    } catch (err) {
      return out;
    }
    const map: Record<string, JFilterGroup[]> | undefined = j.filters;
    if (map === undefined) {
      return out;
    }
    const groups: JFilterGroup[] | undefined = map[tid];
    if (groups === undefined || groups === null) {
      return out;
    }
    for (let i = 0; i < groups.length; i++) {
      const src: JFilterGroup = groups[i];
      const g = new FilterGroup();
      g.key = src.key === undefined ? '' : src.key;
      g.name = src.name === undefined ? g.key : src.name;
      const vs: JFilterValue[] | undefined = src.value;
      if (vs !== undefined) {
        for (let k = 0; k < vs.length; k++) {
          const sv: JFilterValue = vs[k];
          const v = new FilterValue();
          v.n = sv.n === undefined ? '' : sv.n;
          v.v = sv.v === undefined ? '' : sv.v;
          if (v.n.length > 0) {
            g.values.push(v);
          }
        }
```

```
      }
      if (g.key.length > 0 && g.values.length > 0) {
        out.push(g);
      }
    }
    return out;
  }
}
export function mapClasses(list: JClass[]): Category[] {
  const out: Category[] = [];
  for (let i = 0; i < list.length; i++) {
    const c = new Category();
    c.id = String(list[i].type_id === undefined ? '' : list[i].type_id);
    c.name = Cms.s(list[i].type_name);
    out.push(c);
  }
  return out;
}
// ============ 站点 ext 的原始结构（只取筛选那一段） ============
interface JFilterValue {
  n?: string;
  v?: string;
}
interface JFilterGroup {
  key?: string;
  name?: string;
  value?: JFilterValue[];
}
interface JExt {
  filters?: Record<string, JFilterGroup[]>;
}
// ================================================================================
// 文件：entry/src/main/ets/common/Device.ets     （共 83 行）
// ================================================================================
/**
 * 设备形态识别 —— 让同一套 UI 在「手机 / 电视」上各自适配。
 *
 * 判据：
 *   1. deviceInfo.deviceType 是官方分类，取值 'phone' | 'tv' | 'tablet' | '2in1'
| 'wearable' | 'default'。
 *      手机（含折叠屏 Pura X）返回 'phone'，电视返回 'tv'，这是主判据。
 *   2. 兜底：有些固件/模拟器返回 'default' / 'tablet'，此时用屏幕短边（vp）猜：
 *      短边 < 600vp 视为手机。电视横屏大屏、平板大屏都不落在这个区间。
 *
 * 结果写进 AppStorage（全局单例，跨页面共享）：
 *   isPhone : boolean  —— 手机布局（底部 Tab + 网格 + 触摸）
 *   isTv    : boolean  —— 电视布局（顶部功能行 + 横向轨道 + 遥控器焦点）
 *
 * 页面用 @StorageProp('isPhone') isPhone: boolean = false 读取。
 * Device.init() 必须在 loadContent 之前调用（放 EntryAbility.onCreate 最前面），
```

```
 * 这样首页第一次 build 就能读到正确值，不会闪一帧错误的布局。
 */
import deviceInfo from '@ohos.deviceInfo';
import display from '@ohos.display';
/** AppStorage 键名，页面用 @StorageProp 绑定 */
export const KEY_IS_PHONE: string = 'isPhone';
export const KEY_IS_TV: string = 'isTv';
export class Device {
  /** 入口初始化，必须在 EntryAbility.onCreate 里最先调用 */
  static init(): void {
    let dt: string = 'default';
    try {
      dt = deviceInfo.deviceType;
    } catch (e) {
      dt = 'default';
    }
    let isPhone: boolean = dt === 'phone';
    let isTv: boolean = dt === 'tv';
    // 官方分类拿不准时（default / tablet / 2in1 / wearable），用屏幕短边兜底
    if (dt !== 'phone' && dt !== 'tv') {
      try {
        const d = display.getDefaultDisplaySync();
        const short: number = Math.min(d.width, d.height);
        if (short > 0 && short < 600) {
          isPhone = true;
          isTv = false;
        }
      } catch (e) {
        // 拿不到屏幕信息就维持默认（按电视处理）
      }
    }
    AppStorage.setOrCreate<boolean>(KEY_IS_PHONE, isPhone);
    AppStorage.setOrCreate<boolean>(KEY_IS_TV, isTv);
  }
  /** 是否是手机（含折叠屏）。UI 层优先用 @StorageProp 读，这里给非 UI 代码用 */
  static isPhone(): boolean {
    return AppStorage.get<boolean>(KEY_IS_PHONE) === true;
  }
  /** 是否是电视 */
  static isTv(): boolean {
    return AppStorage.get<boolean>(KEY_IS_TV) === true;
  }
  /**
   * 弹窗/浮层/面板宽度：手机竖屏用相对宽度留边（固定宽度会超出屏宽被裁切），
   * 电视端保持固定宽度不变。isPhone 在启动时已定、运行期不变，静态读取即可。
   */
  static panelW(base: number): string | number {
    return Device.isPhone() ? '92%' : base;
  }
  /** 手机竖屏时留出状态栏/安全区后的可用宽度（vp）下限，供需要精确计算的场景用 */
```

```
  static phoneW(): number {
    try {
      return Math.min(display.getDefaultDisplaySync().width, display.getDefaultDis
playSync().height);
    } catch (e) {
      return 440;
    }
  }
}
// ================================================================================
// 文件：entry/src/main/ets/common/Feedback.ets     （共 105 行）
// ================================================================================
/**
 * 全局点击反馈（声音 + 震动）。
 *
 * 用户按任意按钮时给一个「声音 + 震动」的即时反馈，让操作有实感。
 * 两个通道各自独立开关，偏好存本机 preferences：
 *   tapSound   0 = 关 / 1 = 开（默认开）
 *   tapVibrate 0 = 关 / 1 = 开（默认开）
 *
 * 声音用 SoundPool（低延迟短音效，播放 28ms 的 click.wav，混音模式不打断正在播的
片），
 * 震动用 vibrator 触发一次 20ms 短震。二者都 fire-and-forget，
 * 不阻塞 UI 线程，点击响应依然跟手。
 *
 * 用法（任何页面）：
 *   import { Feedback } from '../common/Feedback';
 *   .onClick(() => { Feedback.tap(); this.xxx(); })
 *
 * 启动时 EntryAbility.onCreate 调 Feedback.init(this.context) 加载音效；
 * 设置页切换开关后调 Feedback.refresh() 刷新内存缓存。
 */
import vibrator from '@ohos.vibrator';
import soundPool from '@ohos.multimedia.soundPool';
import audio from '@ohos.multimedia.audio';
import common from '@ohos.app.ability.common';
import { Store } from './Store';
export class Feedback {
  private static sp: soundPool.SoundPool | null = null;
  private static soundId: number = -1;
  private static ready: boolean = false;
  private static soundOn: boolean = true;
  private static vibrateOn: boolean = true;
  private static inited: boolean = false;
  /** 启动时调用一次（EntryAbility.onCreate），传入 context 用于加载 rawfile 音效
*/
  static init(ctx?: common.UIAbilityContext): void {
    if (Feedback.inited) {
      return;
    }
```

```
    Feedback.inited = true;
    Feedback.refresh();
    if (ctx !== undefined) {
      Feedback.loadSound(ctx);
    }
  }
  private static loadSound(ctx: common.UIAbilityContext): void {
    try {
      const info: audio.AudioRendererInfo = {
        usage: audio.StreamUsage.STREAM_USAGE_MUSIC,
        rendererFlags: 0
      };
      Feedback.sp = soundPool.createSoundPool(1, info);
      ctx.resourceManager.getRawFd('click.wav').then((fd) => {
        if (Feedback.sp === null) {
          return;
        }
        Feedback.sp.load(fd.fd, fd.offset, fd.length).then((id: number) => {
          Feedback.soundId = id;
          Feedback.ready = true;
        }).catch(() => {
        });
      }).catch(() => {
      });
    } catch (e) {
      // 音效加载失败不影响震动反馈
    }
  }
  /** 开关变更后刷新内存缓存（设置页切换开关时调用） */
  static refresh(): void {
    Store.tapSound().then((v: number) => {
      Feedback.soundOn = v !== 0;
    });
    Store.tapVibrate().then((v: number) => {
      Feedback.vibrateOn = v !== 0;
    });
  }
  /** 点一下的反馈（任何按钮 onClick 开头调用） */
  static tap(): void {
    if (Feedback.vibrateOn) {
      try {
        vibrator.vibrate(20);
      } catch (e) {
        // 忽略：个别设备无马达或权限受限
      }
    }
    if (Feedback.soundOn && Feedback.ready && Feedback.sp !== null && Feedback.sou
ndId >= 0) {
      try {
        const params: soundPool.PlayParameters = {
```

```
          loop: 0,
          rate: 1,
          leftVolume: 0.7,
          rightVolume: 0.7,
          priority: 0
        };
        Feedback.sp.play(Feedback.soundId, params).catch(() => {
        });
      } catch (e) {
        // 忽略
      }
    }
  }
}
// ======================================================================
// 文件: entry/src/main/ets/common/Http.ets     (共 187 行)
// ======================================================================
import http from '@ohos.net.http';
const UA: string = 'Mozilla/5.0 (Linux; HarmonyOS; SmartScreen) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36';
class Resp {
  code: number = 0;
  body: string = '';
  location: string = '';
}
/** 精简 HTTP 客户端: GET + 手写重定向跟随 + 中文错误提示 */
export class Http {
  static async text(url: string, extra?: Record<string, string>, timeout?: number)
: Promise<string> {
    const ms: number = timeout === undefined ? 12000 : timeout;
    let cur: string = url;
    let last: Resp = new Resp();
    for (let hop = 0; hop < 3; hop++) {
      last = await Http.once(cur, extra, ms);
      if (last.code >= 300 && last.code < 400 && last.location.length > 0) {
        cur = Http.abs(cur, last.location);
        continue;
      }
      break;
    }
    if (last.code === 0) {
      return last.body;
    }
    if (last.code >= 400) {
      let hint: string = '';
      if (last.code === 403) {
        hint = ', 被站点拒绝 (可能需要 Referer 或已限制地区)';
      } else if (last.code === 429) {
        hint = ', 请求过于频繁被限流';
      } else if (last.code === 404) {
```

```
        hint = '，接口不存在（数据源可能已失效）';
      }
      throw new Error('HTTP ' + last.code + hint);
    }
    return last.body;
  }
  /**
   * 只抓首包（前 4KB），用来判断一个流地址「活不活」和「快不快」。
   *
   * 为什么不直接用 text()：直播地址 302 到 mp4 时会整段下载，几秒都回不来。
   * 这里带 Range 头，读超时一到就 destroy，拿多少算多少。
   * 抛异常 = 连不上；返回空串 = 服务端给了 4xx/5xx。
   */
  static async peek(url: string, ms: number): Promise<string> {
    const req = http.createHttp();
    try {
      const opt: http.HttpRequestOptions = {
        method: http.RequestMethod.GET,
        header: {
          'User-Agent': UA,
          'Accept': '*/*',
          'Range': 'bytes=0-4095'
        },
        connectTimeout: ms,
        readTimeout: ms,
        expectDataType: http.HttpDataType.STRING,
        usingCache: false
      };
      const res = await req.request(url, opt);
      if (res.responseCode >= 400) {
        return '';
      }
      if (typeof res.result !== 'string') {
        return '#BIN';
      }
      const s: string = res.result as string;
      return s.length > 4096 ? s.substring(0, 4096) : s;
    } catch (err) {
      const e = err as Error;
      throw new Error('连接失败：' + e.message);
    } finally {
      req.destroy();
    }
  }
  /**
   * WebDAV PROPFIND：列目录 / 探测连接。
   *
   * http.RequestMethod 枚举只有 OPTIONS/GET/HEAD/POST/PUT/DELETE/TRACE/CONNECT
   * 八个，没有 PROPFIND；但底层网络栈走的是 libcurl 语义（CURLOPT_CUSTOMREQUEST）
'
```

```
   * 接受任意 method 字符串。这里用双重断言把 'PROPFIND' 塞进去。
   * Depth: 1 表示列当前目录的直接子项（不递归）。
   */
  static async propfind(url: string, extra: Record<string, string> | undefined, ti
meout?: number): Promise<string> {
    const ms: number = timeout === undefined ? 15000 : timeout;
    const req = http.createHttp();
    try {
      const header: Record<string, string> = { 'User-Agent': UA, 'Depth': '1' };
      if (extra !== undefined) {
        const keys: string[] = Object.keys(extra);
        for (let i = 0; i < keys.length; i++) {
          header[keys[i]] = extra[keys[i]];
        }
      }
      const opt: http.HttpRequestOptions = {
        method: 'PROPFIND' as http.RequestMethod,
        header: header,
        connectTimeout: ms,
        readTimeout: ms,
        expectDataType: http.HttpDataType.STRING,
        usingCache: false
      };
      const res = await req.request(url, opt);
      if (res.responseCode >= 400) {
        let hint: string = '';
        if (res.responseCode === 401) {
          hint = '，用户名或密码错误';
        } else if (res.responseCode === 403) {
          hint = '，无权限访问该路径';
        } else if (res.responseCode === 404) {
          hint = '，路径不存在';
        }
        throw new Error('WebDAV HTTP ' + res.responseCode + hint);
      }
      if (typeof res.result === 'string') {
        return res.result as string;
      }
      return '';
    } catch (err) {
      const e = err as Error;
      if (e.message.indexOf('WebDAV HTTP') === 0) {
        throw e;
      }
      throw new Error('WebDAV 请求失败：' + e.message);
    } finally {
      req.destroy();
    }
  }
  private static async once(url: string, extra: Record<string, string> | undefined
```

```
, ms: number): Promise<Resp> {    const req = http.createHttp();
    const r = new Resp();
    try {
      const header: Record<string, string> = { 'User-Agent': UA, 'Accept': '*/*' }
;
      if (extra !== undefined) {
        for (const k of Object.keys(extra)) {
          header[k] = extra[k];
        }
      }
      const opt: http.HttpRequestOptions = {
        method: http.RequestMethod.GET,
        header: header,
        connectTimeout: ms,
        readTimeout: ms,
        expectDataType: http.HttpDataType.STRING,
        usingCache: false
      };
      const res = await req.request(url, opt);
      r.code = res.responseCode;
      if (typeof res.result === 'string') {
        r.body = res.result as string;
      }
      const h = res.header as Record<string, string>;
      const keys: string[] = Object.keys(h);
      for (let i = 0; i < keys.length; i++) {
        if (keys[i].toLowerCase() === 'location') {
          r.location = String(h[keys[i]]);
        }
      }
    } catch (err) {
      const e = err as Error;
      throw new Error('网络请求失败：' + e.message);
    } finally {
      req.destroy();
    }
    return r;
  }
  private static abs(base: string, loc: string): string {
    if (loc.startsWith('http://') || loc.startsWith('https://')) {
      return loc;
    }
    const i: number = base.indexOf('://');
    if (i < 0) {
      return loc;
    }
    const slash: number = base.indexOf('/', i + 3);
    if (loc.startsWith('/')) {
      return slash < 0 ? base.substring(0, base.length) + loc : base.substring(0,
slash) + loc;
```

```
    }
    return slash < 0 ? base + '/' + loc : base.substring(0, slash + 1) + loc;
  }
}
// ============================================================================
// 文件：entry/src/main/ets/common/LiveProbe.ets     （共 228 行）
// ============================================================================
/**
 * 播放源测速排行 + 可用性检测。
 *
 * 为什么抽成公共模块：这套逻辑原先只写在设置页里，直播页拿不到，于是用户在
 * 直播页看到某个源「一堆台但全点不开」，只能自己退回设置页去猜哪个源好用。
 * 现在两处共用同一份实现，结论也一致（同一套超时、抽样数、评分权重）。
 *
 * 两类检测，回答的是两个不同问题：
 *   1. 源级 rankOne —— 这个源好不好？拉列表快不快、有多少台、抽样能不能播。
 *      用于「测速排行」，用来在多个源之间挑一个。
 *   2. 频道级 channels —— 这个源里哪些台是死的？逐个抓首包，
 *      用于把失效台标灰，避免用户点一次等 20 秒。
 *
 * 一个必须记住的教训：只看「列表拉得快」完全没用。实测有源 74 ms 就拉完
 * 500 多个台，抽样一测全连不上。所以可播率必须实测，不能靠速度推断。
 */
import { Live } from './Live';
import { Channel, ChannelGroup } from './Model';
/** 一次源级测速的结论 */
export class LiveScore {
  name: string = '';
  url: string = '';
  /** 列表是否拉得下来（不代表频道能播） */
  ok: boolean = false;
  /** 拉完整份列表的耗时（ms），-1 = 没测出来 */
  ms: number = -1;
  /** 频道总数 */
  count: number = 0;
  /** 抽样里能连上的数量 */
  alive: number = 0;
  /** 实际抽样数量（源里台太少时可能小于计划值） */
  sampled: number = 0;
  /** 综合评分 0~100，越大越好 */
  score: number = 0;
  /** 给界面直接用的一句话，避免每处各拼一遍 */
  state: string = '';
}
export class LiveProbe {
  /** 每个源抽样几个台实测。3 个够看出「全军覆没」，再多会明显拖慢排行 */
  static readonly SAMPLE: number = 3;
  /** 单条流等首包的超时。死链常常直接挂住不返回，超过这个时间就当失效 */
  static readonly STREAM_MS: number = 3000;
  /** 拉列表的超时。8 秒拉不下来说明这个源本身有问题 */
```

```
  static readonly LIST_MS: number = 8000;
  /** 外层兜底超时：Http 的 abort 不是所有情况都能打断，外层再压一道 */
  static readonly GUARD_MS: number = 10000;
  /**
   * 测一个源。不会抛异常，失败也会返回一个 ok=false 的结果。
   *
   * 顺序是刻意的：先把列表拉下来（拿到频道数），再抽样测流。
   * 反过来做的话，列表都拉不下来的源还要白等 3 次流超时。
   */
  static async rankOne(name: string, url: string): Promise<LiveScore> {
    const r = new LiveScore();
    r.name = name;
    r.url = url;
    const t0: number = Date.now();
    let gs: ChannelGroup[] = [];
    try {
      gs = await Promise.race([Live.load(url, LiveProbe.LIST_MS), LiveProbe.guard(
LiveProbe.GUARD_MS)]);
    } catch (e) {
      r.state = '拉取失败';
      return r;
    }
    r.ms = Date.now() - t0;
    let n: number = 0;
    for (let i = 0; i < gs.length; i++) {
      n += gs[i].channels.length;
    }
    if (n === 0) {
      r.state = '列表为空';
      return r;
    }
    r.ok = true;
    r.count = n;
    // 抽样：先从不同分组里各取第一个（避免全落在同一个台的多条线路上），
    // 分组数不够再回头补该组里的第 2、3 个 —— 有的源只有一个分组装着四百个台，
    // 只从这里取一个样本，「可播 1/1」的说服力太弱，一个台抽风就会把整个源判死。
    const samples: string[] = [];
    for (let i = 0; i < gs.length && samples.length < LiveProbe.SAMPLE; i++) {
      const chs: Channel[] = gs[i].channels;
      if (chs.length > 0) {
        samples.push(chs[0].urls.length > 0 ? chs[0].urls[0] : chs[0].url);
      }
    }
    for (let k = 1; samples.length < LiveProbe.SAMPLE; k++) {
      let added: boolean = false;
      for (let i = 0; i < gs.length && samples.length < LiveProbe.SAMPLE; i++) {
        const chs: Channel[] = gs[i].channels;
        if (k < chs.length) {
          const u: string = chs[k].urls.length > 0 ? chs[k].urls[0] : chs[k].url;
          if (samples.indexOf(u) < 0) {
```

```
            samples.push(u);
            added = true;
          }
        }
      }
      if (!added) {
        break;      // 全源就那么几个台，补不出来
      }
    }
    r.sampled = samples.length;
    const jobs: Promise<boolean>[] = [];
    for (let i = 0; i < samples.length; i++) {
      jobs.push(Live.probe(samples[i], LiveProbe.STREAM_MS));
    }
    const rs: boolean[] = await Promise.all(jobs);
    for (let i = 0; i < rs.length; i++) {
      if (rs[i]) {
        r.alive++;
      }
    }
    r.score = LiveProbe.score(r);
    r.state = r.ms + ' ms · ' + r.count + ' 台 · 可播 ' + r.alive + '/' + r.sample
d;
    return r;
  }
  /**
   * 评分：可播率占 70%，速度占 30%。
   *
   * 权重不对称是有意的 —— 直播首要的是「点开能看」。一个 200 ms 但半数是死台的源
,
   * 实际体验远不如一个 800 ms 但全部能播的源。速度按 50 ms 折 1 分递减：
   * 100 ms → 98 分，1000 ms → 80 分，5000 ms → 0 分。
   */
  static score(r: LiveScore): number {
    if (!r.ok) {
      return 0;
    }
    const availability: number = r.sampled > 0 ? (r.alive / r.sampled) * 100 : 0;
    const speed: number = Math.max(0, 100 - r.ms / 50);
    return Math.round(availability * 0.7 + speed * 0.3);
  }
  /**
   * 批量测速，按批推进（每批 3 个）。
   *
   * 并发全部拉满的话，同时开十几个 HTTP 连接，弱网下会互相拖慢，
   * 测出来的耗时反而不准（本来 300 ms 的源测成 2 秒）。
   */
  static async rankAll(items: LiveScore[], onEach: (i: number, r: LiveScore) => vo
id): Promise<LiveScore[]> {
    const out: LiveScore[] = [];
```

```
    for (let i = 0; i < items.length; i++) {
      const r = new LiveScore();
      r.name = items[i].name;
      r.url = items[i].url;
      out.push(r);
    }
    for (let s = 0; s < out.length; s += 3) {
      const batch: Promise<void>[] = [];
      for (let i = s; i < Math.min(s + 3, out.length); i++) {
        const idx: number = i;
        batch.push((async (): Promise<void> => {
          const r = await LiveProbe.rankOne(out[idx].name, out[idx].url);
          out[idx] = r;
          onEach(idx, r);
        })());
      }
      await Promise.all(batch);
    }
    return out;
  }
  /** 按评分排序；同分时台多的排前面，再同则按耗时 */
  static sort(list: LiveScore[]): LiveScore[] {
    const out: LiveScore[] = list.slice();
    out.sort((a: LiveScore, b: LiveScore): number => {
      if (a.score !== b.score) {
        return b.score - a.score;
      }
      if (a.ok !== b.ok) {
        return a.ok ? -1 : 1;
      }
      if (a.count !== b.count) {
        return b.count - a.count;
      }
      return a.ms - b.ms;
    });
    return out;
  }
  /**
   * 逐个检测频道能不能连上，返回失效数量。
   *
   * 并发控制在 4：再高容易把源站当攻击，几十个台同时探活会被限流，
   * 结果是一堆「假失效」——明明能播的台被标灰，比不标更糟。
   *
   * onTick 每检测完一条调一次，界面可以边测边标灰，不用等全部结束。
   */
  static async channels(chs: Channel[], onTick: (ch: Channel, alive: boolean) => v
oid,
    parallel: number = 4): Promise<number> {
    let dead: number = 0;
    for (let s = 0; s < chs.length; s += parallel) {
```

```
      const jobs: Promise<void>[] = [];
      for (let i = s; i < Math.min(s + parallel, chs.length); i++) {
        const ch: Channel = chs[i];
        const u: string = ch.urls.length > 0 ? ch.urls[0] : ch.url;
        jobs.push((async (): Promise<void> => {
          const ok: boolean = await Live.probe(u, LiveProbe.STREAM_MS);
          if (!ok) {
            dead++;
          }
          onTick(ch, ok);
        })());
      }
      await Promise.all(jobs);
    }
    return dead;
  }
  /** 外层兜底超时：到点就抛，让 rankOne 走到「拉取失败」分支 */
  private static guard(ms: number): Promise<ChannelGroup[]> {
    return new Promise<ChannelGroup[]>((_, reject: (e: Error) => void) => {
      setTimeout(() => {
        reject(new Error('探测超时'));
      }, ms);
    });
  }
}
// ======================================================================
// 文件：entry/src/main/ets/common/PhoneTabBar.ets     （共 69 行）
// ======================================================================
/**
 * 手机端底部 Tab 导航 —— 全局共享，供首页和各子页面复用。
 *
 * 电视端没有这个：电视用顶部功能行 + 遥控器焦点导航。
 * 手机端首页和各功能页底部都放同一个栏，用户不用回首页就能切换页面。
 *
 * active 传「当前页面的路由」，对应项高亮、点击不重复跳转（避免页面无限堆叠）；
 * 首页传空字符串（首页不属于任何子页面 Tab）。
 */
import router from '@ohos.router';
import { T } from './Model';
function tabIcon(label: string): Resource {
  if (label === '直播') {
    return $r('app.media.ic_live');
  }
  if (label === '搜索') {
    return $r('app.media.ic_search');
  }
  if (label === '收藏') {
    return $r('app.media.ic_collect');
  }
  if (label === '设置') {
```

```
    return $r('app.media.ic_setting');
  }
  return $r('app.media.ic_vod');
}
/** 单个 Tab 项：图标 + 文字，整块可点 */
@Builder
function TabItem(label: string, page: string, active: string) {
  Column() {
    Image(tabIcon(label))
      .width(20)
      .height(20)
      .objectFit(ImageFit.Contain)
    Text(label)
      .fontSize(11)
      .fontColor(page === active ? T.SELECTED : T.SUB)
      .margin({ top: 3 })
      .maxLines(1)
  }
  .layoutWeight(1)
  .height('100%')
  .justifyContent(FlexAlign.Center)
  .onClick(() => {
    if (page !== active) {
      router.pushUrl({ url: page });
    }
  })
}
/** 底部 Tab 栏：5 个入口，触屏拇指热区 */
@Builder
export function PhoneTabBar(active: string) {
  Row() {
    TabItem('点播', 'pages/VodPage', active)
    TabItem('直播', 'pages/LivePage', active)
    TabItem('搜索', 'pages/Search', active)
    TabItem('收藏', 'pages/CollectPage', active)
    TabItem('设置', 'pages/SettingPage', active)
  }
  .width('100%')
  .height(54)
  .padding({ left: 4, right: 4 })
  .backgroundColor(T.SURFACE)
  .border({ width: { top: 1 }, color: '#1FFFFFFF' })
}
// ============================================================================
// 文件：entry/src/main/ets/common/Pinyin.ets     （共 140 行）
// ============================================================================
/**
 * 汉字 → 拼音首字母（声母）映射，供搜索联想做「拼音首字母」猜测。
 *
 * 数据来源：GB2312 字库（一、二级共 6763 个汉字），用 macOS
```

```
 * CFStringTransform 批量转拼音后取首字母；另对 21 个影视语境下
 * 首字母易分歧的多音字（长/重/传/乐/行…）做了双声母修正，
 * 生成缩写时这类字会分叉出多个候选，保证「cz」也能命中「长征」。
 *
 * 说明：这是本地纯内存查表，不联网；表外字（超 GB2312 的生僻字）
 * 会被跳过，剩余字仍能提供足够首字母用于匹配。
 */
const MAIN: string = 'a:啊阿埃挨哎唉哀皑癌蔼矮艾碍爱隘鞍氨安俺按暗岸胺案肮昂盎凹敖熬翱祆傲奥懊澳谙坳埯拗捱搐吖嗷嗳嗌岙犴庵廒遨媪嫒鹜璎桉葵暧砹铵锕锬鹌聱螯霭鏊鳌鏖黯;b:芭捌扒叭吧笆八疤巴拔跋靶把耙坝霸罢爸白柏百摆佰败拜稗斑班搬扳般颁板版扮拌伴瓣半办绊邦帮梆榜膀绑棒磅蚌镑傍谤苞胞包褒剥薄雹保堡饱宝抱报暴豹鲍爆杯碑悲卑北辈背贝钡倍狈备惫焙被奔苯本笨崩绷甭泵蹦迸逼鼻比鄙笔彼碧蓖蔽毕毙毖币庇痹闭敝弊必壁臂避陛鞭边编贬扁便变卞辨辩辫遍标彪膘表鳖憋别瘪彬斌濒滨宾摈兵冰柄丙秉饼炳病并玻菠播拨钵波博勃搏铂箔伯帛舶脖膊渤驳捕卜哺补埠不布步簿部怖埔匕孛匾俾傧勹亳禀冫阪邢邶弁畚坌坂芘芣芟苹萍蒎菠葡葆蓓蒡蕃薛捭摒卟吡呗哔啵嘣岜豳猃饽庫忭悖愎汴浜滹濞灞逋弼妣嫉婢嬖孢骠缏缤玢璧杓槟檗殡瓿髡晡贲掰擘膑飏飚飚煲煸灬砭碚碥碡畀钎钣钯钸铋锛镔镖镳秕鸨鹁鹌瘢瘭癍窦裼裨裨褓褊蝙笠箪算篦簸舭舨襞粑醭蹩趵跛跸踣龅鏊鲅鳊鳔鞴髀髌魅彭鬓;c:擦猜裁材才财睬踩采彩菜蔡餐参蚕残惭惨灿苍舱仓沧操糙槽曹草厕策侧册测层蹭插叉茬茶查碴搽察岔差诧拆柴豺搀掺蝉馋谗缠铲产阐颤昌猖场尝常偿肠厂敞畅唱倡超抄钞嘲潮巢吵炒车扯撤掣彻澈郴臣辰尘晨忱沉陈趁衬撑称城橙成呈乘程惩澄诚承逞骋秤吃痴持池迟弛驰耻齿侈尺赤翅斥炽充冲虫崇宠抽酬畴踌稠愁筹仇绸瞅丑臭初出橱厨躇锄雏滁除楚础储矗搐触处揣川穿椽船喘串疮窗幢床闯创吹炊捶锤垂春椿醇唇淳纯蠢戳绰疵茨磁雌辞慈瓷词此刺赐次聪葱囱匆从丛凑粗醋簇促蹿篡窜摧崔催脆瘁粹淬翠村存寸磋撮搓措挫错伺畜亍丞厝伥伧侪俦傺汆冁谄谌谶陲刍鬯坼埕墀艹茬苁茬莅茏茴菖萃蒇蔟抻摅撺叱咄咻啐啜啻嗔嗤嘈嚓帱岑嵯彳徂徜猝猹舛饧馇怵忏忡怅怆怵怊恻恼惆悴憧憬阊汊涔淙漕潺澶宸辶遄孱羼屮姹娼婵媸嫦骢骢骣绌巛琮琛璀璁璨杈杵枨枞枞柽楮棰槌榇槎榱樗檫殂辍辏晁晟觇毳氚敕脞腠塍膪焯爨祠禅砗碜曚瞠钏钗铖铳锉锸镡镩镲矬鸱鹑鹋瘁瘥瘛瘳衩裎褚褫皴耖蚤蚩蛏蜍蝽蝻螭螬蟾呓嗳啻啧嘧
```

贺嘿黑痕很狠恨哼亨横衡恒轰哄烘虹鸿洪宏弘红喉侯猴吼厚候后呼乎忽瑚壶葫胡蝴狐糊湖弧虎唬护互沪户花哗华猾滑画划化话槐徊怀淮坏欢环桓缓换患唤痪豢焕涣宦幻荒慌黄磺蝗簧皇凰惶煌晃幌恍谎灰挥辉徽恢蛔回毁悔慧卉惠晦贿秽会烩汇讳诲绘荤昏婚魂浑混豁活伙火获或惑霍货祸咳剿黉訇沍讧诃诙诨隍邗郇奂劾壑堠茴荟茇萑菡蒿蕙蕨薨薅藿蘅撖擐攉哎哕嗯喙嘀嗥嗨嘻嚄囫圜岵後徨猢獾夥馁怙愦隳闵阁阖沆泓洹洄洽浒浣湟溷潇溥湛濠瀚灏寰逅逭遑彗骅纡绡缋缨珩珲琥璜桀桁橱轷昊曷晖晗肓胲觳烝焓煳舀扈祜恚耆盍钬铓铪锶锾镬皓瓠锼瘥癀耛顸颃颌颔颢卢虺蚌蚶蚵蟥蟪蠖笏篁篌糇翮醐醢踝斛蝗鳇骷鬟麾鼾;j:击圾基机畸稽积箕肌饥迹激讥鸡姬绩缉吉极棘辑籍集及急疾汲即嫉级挤几脊己蓟技冀季伎祭剂悸济寄寂计记既忌际妓继纪嘉枷夹佳家加荚颊贾甲钾假稼价架驾嫁歼监坚尖笺间煎兼肩艰奸缄茧检柬碱硷拣捡简俭剪减荐鉴践贱见键箭件健舰剑饯渐溅涧建僵姜将浆江疆蒋桨奖讲匠酱蕉椒礁焦胶交郊浇骄娇嚼搅铰矫侥脚狡角饺缴绞剿教酵轿较叫窖揭接皆秸街阶截劫节桔杰捷睫竭洁结姐戒藉芥界借介疥诫届巾筋斤金今津襟紧锦仅谨进靳晋禁近烬浸尽劲荆兢茎睛晶鲸京惊精粳经井警景颈静境敬镜径痉靖竟竞净炯窘揪究纠玖韭久灸九酒厩救旧臼舅咎就疚鞠拘狙疽居驹菊局咀矩举沮聚拒据巨具距踞锯俱句惧炬剧捐鹃娟倦眷卷绢撅攫抉掘倔爵觉决诀绝均菌钧军君峻俊竣浚郡骏茄丌亟乩厥刭刽劂冂伽佶佼倨偈徼僭憔傲讦讵诂谏谪谮卩卺阱郑郫矍蹩芄芟芰苣苴莒荞茭茫蒉菁堇菅葭瘁蒹蕨蔻拮挢捃掎掬搀叽咭哜唧喈啾嗟嚯噍噤囝岌岬峤崛嵴徼狷獍獗馑廛憬阄泾浃洎洚涓湫湔蹇謇迥迦迳遽彐屐屣驾妗姣婧婕孑孓骥绛缙缣畿玑珏珈琚瑾枧柩桕桀桑桷楗椐楫榘檫橛橘殛戛戛戢戬炅赙赞觊觐垒墨犄犋毽毂氍毡腱毽敫胼胛胫腈腱飓蔺旌燔扃恕矶碣礓睑羁蠲钅 钜铗锏锔锗镌镓镢嵇稷皎鸠鹣鹪鹜疖痂瘕瘠窭衿裯裾皱矜耩颉虮蚴蛱蛟笈笄笕衿裯裾皱矜耩颉虮蚴蛱蛟笈笄笕阿腭旷皖恪悝惺悃愦闶阃阚溘逵尻骒纩绔缫珂琨栲轲戡暌贶犊氪肼脍龛眍瞵瞅钪钶铐铠铿锞锞锟锴稞疴窠裉聩颍蛞蝰蝌筘笆篑赝醌跬鲲骷髁髋髡;l:垃拉喇蜡腊辣啦莱来赖蓝婪栏拦篮阑兰澜谰揽览懒缆烂滥琅榔狼廊郎朗浪捞劳牢老佬姥酪烙涝勒雷镭蕾磊累儡垒擂肋类泪棱楞冷厘梨犁黎篱狸离漓理李里鲤礼莉荔吏栗丽厉励砾历利傈例俐痢立粒沥隶力璃哩俩联莲连镰廉怜涟帘敛脸链恋炼练粮凉梁粱良两辆量晾亮谅撩聊僚疗燎寥辽潦了撂镣廖料列裂烈劣猎琳林磷霖临邻鳞淋凛赁吝拎玲菱零龄铃伶羚凌灵陵岭领另令溜琉榴硫馏留刘瘤流柳六龙聋咙笼窿隆垄拢陇楼娄搂篓漏陋芦卢颅庐炉掳卤虏鲁麓碌露路赂鹿潞禄录陆戮驴吕铝侣旅履屡缕虑氯律率滤绿峦挛孪滦卵乱掠略抡轮伦仑沦纶论萝螺罗逻锣箩骡裸落洛骆络剌仂俪俚傈偻蔻嬴赢冽诔丽郦坜咙垆坪塄楞芳茏苓荤莅茛蒌蓠蒮蔹蔺蓼藜奁旭捋摞撸叻呖呤咧唠啉啷唳喱喽嘞嘹噜囵囡岚崂峡嵝嶙徕猢猡獠廪愣懔闾阆泖泷泸泠泺洌浏涞渌溧潍潋澈濑澧濂寮逦逯遛遴邋娈娌媛嫘骊骝绫绺缡缥缭珑珞琏璐栅栊栌柃栎栲栾棂椤椋楝榄橹檑檩殆铲铘铙铞铟铖铞铟铖铞铟铖

圃普浦谱曝瀑丕丿叵剽仳俜俳匍裒谝陂陴邳郸鄱圮埤堋鼙苤葩蔗匏拚掊擗哌砻嘌嘭噗噼帔彷狍庇庖怦爿泮湮溍溥滂濮逄姘娉媲嫔嫖骿纰缥珀璞枇杷枰椭殍甓攴牝犏毯气攵胼脬旆睥瞟累钍钜铍锆镁镨皤疱癖袢襻疋鼙蚍蜱螃螵蟛蟠筥筢翩醅蹁蹒蹼貔霩鲆;q:期欺栖戚妻七凄漆柒沏其棋歧畦崎脐齐旗祈祁骑起岂乞企启契砌器气迄弃汽泣讫掐恰洽牵扦钎铅千迁签仟谦乾黔钱钳前潜遣浅谴堑嵌欠歉枪呛腔羌墙蔷强抢橇锹敲悄桥瞧乔侨巧鞘撬翘峭俏窍切且怯窃钦侵亲秦琴勤芹擒禽寝沁青轻氢倾卿清擎晴氰情顷请庆琼穷秋丘邱球求囚酋泅趋蛆曲躯屈驱渠取娶龋趣去圈颧权醛泉全痊拳犬券劝缺炔瘸却鹊榷确雀裙群亓勖俅俟倩佥诎诠诮谯阡邛郄劬凵巯圻芊芑芩芨芪苘茕茜荞荃萁萋葜葺蕖蕲蘧掮掀吣嗪嘁噙圊屹岍岐岖衢犭犰悸悛愜愀憪憔阒阕阙刂戕汔淇溙骞搴蓑衰速逶遒妾嫱骐绮缋缱缲琪琦璩杞棺筑楸樯槭檠樵檎铨饯耆赋觑挈氍肴朐欹炝祛祺悫愆憩硗碛磬磲畎钤铨锾锖锲镅铗鸽癯穹袷裎颀虔虮蚯蛩蛐蛑蜻蜞蜷蜡螓蝼蠼罄筘笙箐篾箝衾裘羟糗綦綮麴趕跄蹊蹊謦瞿蜚鲭鳅鳍鞒髂鬈麒黢黯鼢;r:然燃冉染瓤壤攘嚷让饶扰绕惹热壬仁人忍韧任认刃妊纫扔仍日戎茸蓉荣融熔溶容绒冗揉柔肉茹蠕儒孺如辱乳汝入褥软阮蕊瑞锐闰润若弱亻仞偌苪冉荛荏甚蓐蓯薷嚅嵘狱忏洳溽濡娆缛枘桡榕轫彤肮禳睿铞稔穰衽襦颥蚋蚺蜍箸糅蹂鞣髯;s:匙撒洒萨腮鳃塞赛三叁伞散桑嗓丧搔骚扫嫂瑟色涩森僧莎砂杀刹沙纱傻啥煞筛晒珊苫杉山删煽衫闪陕擅赡膳善汕扇缮墒伤商赏晌上尚裳梢捎稍烧芍勺韶少哨邵绍奢赊蛇舌舍赦摄射慑涉社设砷申呻伸身深娠绅神沈审婶甚肾慎渗声生甥牲升绳盛剩胜圣师失狮施湿诗尸虱十石拾时什食蚀实识史矢使屎驶始式示士世柿事拭誓逝势是嗜噬适仕侍释饰氏市恃室视试收手首守寿授售受瘦兽蔬枢梳殊抒输叔舒淑疏书赎孰熟薯暑曙署蜀黍鼠属术述树束戍竖墅庶数漱恕刷耍摔衰甩帅栓拴霜双爽谁水睡税吮瞬顺舜说硕朔烁斯撕嘶思私司丝死肆寺嗣四似饲巳松耸怂颂送宋讼诵搜艘擞嗽苏酥俗素速粟僳塑溯诉肃酸蒜算虽隋随绥髓碎岁穗遂隧祟孙损笋蓑梭唆缩琐索锁所厦卅啻厍厮剜仨倏佘夙兕凇讪诛谂谇谖谧鄯劭叟勹塾铤坳墉埽芟苔荪莳荽莘菘菽蓄蒴藐薮扌摅搠搡弒咝哂唢哽唰嗖嗦嗦嗄嗍嗾噻崧嵘嵩彡狩狲狻猞飧饣馊馓忪悚愫闩氵汜沭泗涞淞渖涮溲滠澍澌潜潲濉邃妁姒姗姝娑嬗孀骋缡彡纾绡绶缌缫桫犏犏犏

盈影颖硬映哟拥佣臃痈庸雍踊蛹咏泳涌永恿勇用幽优悠忧尤由邮铀犹油游酉有友右佑釉诱又幼迂淤于盂榆虞愚舆余俞逾鱼愉渝渔隅予娱雨屿禹宇语羽玉域芋郁遇喻峪御愈欲狱育誉浴寓裕预豫驭鸳渊冤元垣袁原援辕园员圆猿源缘远苑愿怨院曰约越跃钥岳粤月悦阅耘云郧匀陨允运蕴酝晕韵孕轧禺天爻胤毓庵廇赝卣刈劓伛伢攸佚侑俏俨俣俑偃龠兖赢讠诒谀谒谕谶邺郓郢郾鄢郸辶壅圯垭垠垸埸堙塬墉懿芫芸苡茆莹莠莜荻莺萸菸萦蓥蘋薏弈奕挹挪揠揄掾撄弋吆呓吲呦咦咿唷唷喁喑嘤噫围圄圉岈峄崦崾嵛嶷徉徭狈狻猗夤饮饴馀庾膺怏怿怡恢恽悒愠慵闫阈阙沅泱洇湮滟滠漾瀛瀹宥迂迤鬻妍妪妤娅媛嫣驿纡纭缢幺邕琊珧瑛琐瑜瑗璎韫杳柚椭楹椽檐猷殒殪轶轫昀昱晔晏曜贻觇氩氤氲牖爰刖肴胭腌腴媵欻於旖炀烨烊焱煜熨熘燠恙尹聿砑智眙睚罨钇钰钺铕铘铪铞镛镱甬鸢鹆鹞鹳疣痂痍瘐瘀瘗瘿瘾癔翊窈窬窖窜寇衤颍蚓蚰蚴蛘蜮蜴蝓蝮蝤螈罂竽筵筠筦臾舁舣羿翳舔酏酽雩霪龈龋鼋鲅鳐鳙鞅魇餍黝黯鼬鼖;z:匝砸杂栽哉灾宰载再在咱攒暂赞赃脏葬遭糟凿藻枣早澡蚤躁噪造皂灶燥责择则泽贼怎增憎赠扎喳渣札铡闸眨栅榨咋乍炸诈摘斋宅窄债寨瞻毡詹粘沾盏斩辗崭展蘸栈占战站湛绽樟章彰漳张掌涨杖丈帐账仗胀瘴障招昭找沼赵照罩兆肇召遮折哲蛰辙者锗蔗这浙珍斟真甄砧臻贞针侦枕疹诊震振镇阵蒸挣睁征狰争怔整拯正政帧症郑证芝枝支吱蜘知肢脂汁之织职直植殖执值侄址指止趾只旨纸志挚掷至致置帜峙制智秩稚质炙痔滞治窒中盅忠钟衷终种肿仲众舟周州洲诌粥轴肘帚咒皱宙昼骤珠株蛛朱猪诸诛逐竹烛煮拄瞩嘱主著柱助蛀贮铸筑住注祝驻抓爪拽专砖转撰赚篆桩庄装妆撞壮状椎锥追赘坠缀谆准捉拙卓桌琢茁酌啄着灼浊兹咨资姿滋淄孜紫仔籽滓子自渍字鬃棕踪宗综总纵邹走奏揍租足卒族祖诅阻组钻纂嘴醉最罪尊遵昨左佐柞做作坐座卮丶仄赜仉伫侏倬傯俎冢诏诤诹诼谘谪谮谵阼陟陬郅邾鄯鄹圳堉芷芸茱苺菹蓁蕞奘拶揸摭摺撙擢攥吒咂咤哳唑啧啮啁嘬帙帻幛峥崽嵫嶂徵獐夂馔忮怍惴浈洙浞渚涿潴濯迮彟咫姊妯嫜孳驺驺骓骂纡绉缁缒缜缯缵缪铟璋瓒杼栉柘枳栀桎桢梓棹楱楂楱槠檠樽轵轸轻辄辎臧酝戾眷贽赁贶肫胄胨胗胍朕脤膪旃灶祉祚祇祯禚恣斫砟砦碡磔常眦眵睹钊钲铢铮锃锱镯镲锺雉秣稹鸩鹗鹪疟疰痣瘃瘵瘼宅褶耔颛蚱蛭蜇螽蟑竺笊第笮箏篑箸箴簪籀舯舢舭粢粽糌翥蒌趄趱赭酎酢酯跅踬踯踵躅躜豸觜觯訾龇隹錾鲴鳏鳟髭麈黩';
const EXTRA: string = '藏=cz;长=cz;朝=cz;传=cz;单=ds;还=hh;降=jx;解=jx;乐=ly;奇=qj;区=qo;省=sx;宿=sx;为=ww;相=xx;行=hx;要=yy;应=yy;与=yy;曾=cz;重=cz';
let _map: Map<string, string> | null = null;
/** 构建 字→声母 映射（多音字为多个声母连写，如「长」=「cz」） */
function buildMap(): Map<string, string> {
  if (_map !== null) {
    return _map;
  }
  const m: Map<string, string> = new Map<string, string>();
  const ex: string[] = EXTRA.split(';');
  for (let i = 0; i < ex.length; i++) {
    const p: string = ex[i];
    const eq: number = p.indexOf('=');
    if (eq > 0) {
      m.set(p.substring(0, eq), p.substring(eq + 1));
    }
  }
  const parts: string[] = MAIN.split(';');
  for (let i = 0; i < parts.length; i++) {
    const p: string = parts[i];
    const col: number = p.indexOf(':');
    if (col <= 0) {
      continue;
    }
    const ini: string = p.substring(0, col);
    const chars: string = p.substring(col + 1);
    for (let k = 0; k < chars.length; k++) {

```
// ……（此处省略中间 258 页源码，按登记要求提交前 30 页与后 30 页）……
      this.hoverKey = '';
    }
  }
  build() {
    Stack() {
    Column() {
      Row() {
        Text('‹')
          .fontSize(34)
          .fontColor(this.hi('back') ? T.TXT : T.SUB)
          .width(40)
          .textAlign(TextAlign.Center)
          .focusable(true)
          .defaultFocus(true)
          .onFocus(() => this.hover('back', true))
          .onBlur(() => this.hover('back', false))
          .onHover((hv: boolean) => this.hover('back', hv))
          .onClick(() => {
            Feedback.tap();
            router.back();
          })
        Text('收藏')
          .fontSize(24)
          .fontColor(T.TXT)
          .margin({ left: 8 })
        Text('  ' + this.favs.length + ' 部').fontSize(14).fontColor(T.SUB)
        Blank()
      }
      .width('100%')
      .padding({ left: 24, right: 24, top: 24, bottom: 8 })
      if (this.favs.length === 0) {
        Column() {
          Text('还没有收藏').fontSize(18).fontColor(T.SUB)
          Text('在详情页点「收藏」后会出现在这里')
            .fontSize(15).fontColor(T.DIM).margin({ top: 10 })
        }
        .width('100%').layoutWeight(1).justifyContent(FlexAlign.Center)
      } else {
        Grid() {
          ForEach(this.favs, (f: FavItem) => {
            GridItem() {
              Column() {
                Stack({ alignContent: Alignment.TopEnd }) {
                  Image(f.pic.length > 0 ? f.pic : $r('app.media.app_icon'))
                    .width('100%')
                    .height('100%')
                    .objectFit(ImageFit.Cover)
                    .backgroundColor(T.CARD)
                    .borderRadius({ topLeft: T.RADIUS, topRight: T.RADIUS })
```

```
                  Text('×')
                    .fontSize(16)
                    .fontColor(T.TXT)
                    .backgroundColor(this.hi('x_' + f.siteKey + f.vodId) ? T.BLACK
_80 : T.CARD_HI)
                    .width(26)
                    .height(26)
                    .borderRadius(13)
                    .textAlign(TextAlign.Center)
                    .margin(6)
                    .focusable(true)
                    .onFocus(() => this.hover('x_' + f.siteKey + f.vodId, true))
                    .onBlur(() => this.hover('x_' + f.siteKey + f.vodId, false))
                    .onHover((hv: boolean) => this.hover('x_' + f.siteKey + f.vodI
d, hv))
                    .onClick(() => {
                      Feedback.tap();
                      this.del(f);
                    })
                }
                .width('100%')
                .aspectRatio(16 / 9)
                Text(f.name)
                  .fontSize(16)
                  .fontColor(T.TXT)
                  .backgroundColor(T.CARD)
                  .width('100%')
                  .padding(6)
                  .textAlign(TextAlign.Center)
                  .maxLines(1)
                  .textOverflow({ overflow: TextOverflow.Ellipsis })
                  .borderRadius({ bottomLeft: T.RADIUS, bottomRight: T.RADIUS })
              }
              .width('100%')
              .borderRadius(T.RADIUS)
              .border({ width: T.BORDER, color: this.hi('f_' + f.siteKey + f.vodId
) ? T.TXT : '#00000000' })
              .animation({ duration: T.ANIM, curve: Curve.EaseOut })
              .focusable(true)
              .onFocus(() => this.hover('f_' + f.siteKey + f.vodId, true))
              .onBlur(() => this.hover('f_' + f.siteKey + f.vodId, false))
              .onHover((hv: boolean) => this.hover('f_' + f.siteKey + f.vodId, hv)
)
              .onClick(() => {
                Feedback.tap();
                const p = new DetailParam();
                p.siteKey = f.siteKey;
                p.siteName = f.siteName;
                p.vodId = f.vodId;
                router.pushUrl({ url: 'pages/Detail', params: p });
```

```
            })
          }
        }, (f: FavItem) => 'fav' + f.siteKey + f.vodId)
      }
      .columnsTemplate(this.isPhone ? '1fr 1fr 1fr' : '1fr 1fr 1fr 1fr 1fr 1fr')
      .columnsGap(12)
      .rowsGap(20)
      .width('100%')
      .layoutWeight(1)
      .padding({ left: 24, right: 24, top: 8, bottom: 24 })
      .scrollBar(BarState.Off)
    }
    if (this.isPhone) {
      PhoneTabBar('pages/CollectPage')
    }
  }
  .width('100%')
  .height('100%')
    if (this.tipMsg.length > 0) {
      TipLayer(this.tipMsg)
    }
  }
  .width('100%')
  .height('100%')
  .alignContent(Alignment.Center)
 }
}
// ================================================================================
// 文件：entry/src/main/ets/pages/HistoryPage.ets     （共 263 行）
// ================================================================================
/**
 * 历史页 —— 对照 FongMi 的 activity_history.xml + adapter_vod_list.xml。
 *
 * 原版横条卡：48dp 缩略图 + 名称 16sp + 备注 14sp，整条可点。
 * 这里把备注位换成「看到第几集 + 时间」，信息密度更实用。
 */
import { Feedback } from '../common/Feedback';
import router from '@ohos.router';
import common from '@ohos.app.ability.common';
import { Store, HistItem } from '../common/Store';
import { DetailParam, T } from '../common/Model';
import { Tip, TIP_MSG, TipLayer } from '../common/Tip';
@Entry
@Component
struct HistoryPage {
  @State hist: HistItem[] = [];
  /** 全局轻提示（AppStorage 上的 TIP_MSG），Tip.show() 触发 */
  @StorageLink(TIP_MSG) tipMsg: string = '';
  /** 「清空」的二次确认窗口：3 秒内再点一次才真清 */
  @State confirmClear: boolean = false;
```

```
/** 「删除」单条的二次确认：存当前待确认的 siteKey+vodId，空串表示无待确认 */
@State confirmDel: string = '';
@State hoverKey: string = '';
@State ready: boolean = false;
async aboutToAppear(): Promise<void> {
  await Store.init(getContext(this) as common.UIAbilityContext);
  await this.reload();
  this.ready = true;
}
async onPageShow(): Promise<void> {
  if (this.ready) {
    await this.reload();
  }
}
onBackPress(): boolean {
  router.back();
  return true;
}
async reload(): Promise<void> {
  this.hist = await Store.history();
}
async clearAll(): Promise<void> {
  const n: number = this.hist.length;
  if (n === 0) {
    Tip.show('本来就没有记录');
    return;
  }
  // 二次确认：整表清空不可撤销，遥控器误触代价太大
  if (!this.confirmClear) {
    this.confirmClear = true;
    // 按钮同时变成「确认清空」：光靠底部提示不够，遥控器用户看完提示
    // 再找按钮经常超过 3 秒，窗口一过就白点一次。这里给 6 秒 + 按钮自解释。
    Tip.show('再点一次确认清空（共 ' + n + ' 条），6 秒内有效');
    setTimeout(() => {
      this.confirmClear = false;
    }, 6000);
    return;
  }
  this.confirmClear = false;
  await Store.clearHistory();
  await this.reload();
  Tip.show('已清空 ' + n + ' 条记录');
}
/** 单条删除（二次确认：遥控器误触不可恢复） */
async removeOne(h: HistItem): Promise<void> {
  const key: string = h.siteKey + h.vodId;
  if (this.confirmDel !== key) {
    this.confirmDel = key;
    Tip.show('再点一次确认删除「' + h.name + '」，6 秒内有效');
    setTimeout(() => {
```

```
      if (this.confirmDel === key) {
        this.confirmDel = '';
      }
    }, 6000);
    return;
  }
  this.confirmDel = '';
  await Store.removeHistory(h);
  this.hist = await Store.history();
  Tip.show('已删除「' + h.name + '」');
}
ago(t: number): string {
  if (t <= 0) {
    return '';
  }
  const d: number = Date.now() - t;
  if (d < 60000) {
    return '刚刚';
  }
  if (d < 3600000) {
    return Math.floor(d / 60000) + ' 分钟前';
  }
  if (d < 86400000) {
    return Math.floor(d / 3600000) + ' 小时前';
  }
  return Math.floor(d / 86400000) + ' 天前';
}
hi(key: string): boolean {
  return this.hoverKey === key;
}
hover(key: string, on: boolean): void {
  if (on) {
    this.hoverKey = key;
  } else if (this.hoverKey === key) {
    this.hoverKey = '';
  }
}
build() {
  Stack() {
  Column() {
    Row() {
      Text('‹')
        .fontSize(34)
        .fontColor(this.hi('back') ? T.TXT : T.SUB)
        .width(40)
        .textAlign(TextAlign.Center)
        .focusable(true)
        .defaultFocus(true)
        .onFocus(() => this.hover('back', true))
        .onBlur(() => this.hover('back', false))
```

```
          .onHover((hv: boolean) => this.hover('back', hv))
          .onClick(() => {
            Feedback.tap();
            router.back();
          })
        Text('历史')
          .fontSize(24)
          .fontColor(T.TXT)
          .margin({ left: 8 })
        Text('  ' + this.hist.length + ' 条').fontSize(14).fontColor(T.SUB)
        Blank()
        if (this.hist.length > 0) {
          Text(this.confirmClear ? '确认清空? ' : '清空')
            .fontSize(16)
            .fontColor(this.confirmClear ? T.ERR : T.TXT)
            .height(40)
            .padding({ left: 20, right: 20 })
            .borderRadius(T.RADIUS)
            .textAlign(TextAlign.Center)
            .backgroundColor(this.hi('clr') ? T.CARD_HI : (this.confirmClear ? T.W
HITE_15 : '#00000000'))
            .border({ width: T.BORDER, color: this.hi('clr') || this.confirmClear
? T.TXT : '#00000000' })
            .animation({ duration: T.ANIM, curve: Curve.EaseOut })
            .focusable(true)
            .onFocus(() => this.hover('clr', true))
            .onBlur(() => this.hover('clr', false))
            .onHover((hv: boolean) => this.hover('clr', hv))
            .onClick(() => {
              Feedback.tap();
              this.clearAll();
            })
        }
      }
      .width('100%')
      .padding({ left: 24, right: 24, top: 24, bottom: 8 })
      if (this.hist.length === 0) {
        Column() {
          Text('还没有播放记录').fontSize(18).fontColor(T.SUB)
          Text('看过的影片会出现在这里').fontSize(15).fontColor(T.DIM).margin({ to
p: 10 })
        }
        .width('100%').layoutWeight(1).justifyContent(FlexAlign.Center)
      } else {
        List({ space: 8 }) {
          ForEach(this.hist, (h: HistItem) => {
            ListItem() {
              Row() {
                Image(h.pic.length > 0 ? h.pic : $r('app.media.app_icon'))
                  .width(84)
```

```
          .height(48)
          .objectFit(ImageFit.Cover)
          .borderRadius(T.RADIUS)
          .backgroundColor(T.CARD)
        Column() {
          Text(h.name)
            .fontSize(16)
            .fontColor(T.TXT)
            .maxLines(1)
            .textOverflow({ overflow: TextOverflow.Ellipsis })
          Text((h.epName.length > 0 ? h.epName : '未开始') + '   ·   ' +
            h.siteName + '   ·   ' + this.ago(h.time))
            .fontSize(14)
            .fontColor(T.SUB)
            .maxLines(1)
            .textOverflow({ overflow: TextOverflow.Ellipsis })
            .margin({ top: 4 })
        }
        .alignItems(HorizontalAlign.Start)
        .layoutWeight(1)
        .margin({ left: 16 })
        // 单条删除：独立 onClick，不会冒泡触发整行的「进入详情」
        Text(this.confirmDel === (h.siteKey + h.vodId) ? '确认删除? ' : '
删除')
          .fontSize(14)
          .fontColor(this.confirmDel === (h.siteKey + h.vodId) ? T.ERR : T
.SUB)
          .padding({ left: 12, right: 12, top: 8, bottom: 8 })
          .borderRadius(T.RADIUS)
          .backgroundColor(this.hi('del_' + h.siteKey + h.vodId) ? T.CARD_
HI : '#00000000')
          .border({
            width: T.BORDER,
            color: this.hi('del_' + h.siteKey + h.vodId) ? T.TXT : '#00000
000'
          })
          .focusable(true)
          .onFocus(() => this.hover('del_' + h.siteKey + h.vodId, true))
          .onBlur(() => this.hover('del_' + h.siteKey + h.vodId, false))
          .onHover((hv: boolean) => this.hover('del_' + h.siteKey + h.vodI
d, hv))
          .onClick(() => {
            Feedback.tap();
            this.removeOne(h);
          })
      }
      .width('100%')
      .padding(16)
      .borderRadius(T.RADIUS)
      .backgroundColor(this.hi('h_' + h.siteKey + h.vodId) ? T.CARD_HI : '
```

```
#00000000')
              .border({
                width: T.BORDER,
                color: this.hi('h_' + h.siteKey + h.vodId) ? T.TXT : '#00000000'
              })
              .animation({ duration: T.ANIM, curve: Curve.EaseOut })
              .focusable(true)
              .onFocus(() => this.hover('h_' + h.siteKey + h.vodId, true))
              .onBlur(() => this.hover('h_' + h.siteKey + h.vodId, false))
              .onHover((hv: boolean) => this.hover('h_' + h.siteKey + h.vodId, hv)
)
              .onClick(() => {
                Feedback.tap();
                const p = new DetailParam();
                p.siteKey = h.siteKey;
                p.siteName = h.siteName;
                p.vodId = h.vodId;
                router.pushUrl({ url: 'pages/Detail', params: p });
              })
            }
          }, (h: HistItem) => 'hist' + h.siteKey + h.vodId + h.time)
        }
        .width('100%')
        .layoutWeight(1)
        .padding({ left: 24, right: 24, bottom: 24 })
        .scrollBar(BarState.Off)
      }
    }
    .width('100%')
    .height('100%')
      if (this.tipMsg.length > 0) {
        TipLayer(this.tipMsg)
      }
    }
    .width('100%')
    .height('100%')
    .alignContent(Alignment.Center)
  }
}
// ======================================================================
// 文件: entry/src/main/ets/pages/WebdavPage.ets     (共 759 行)
// ======================================================================
/**
 * WebDAV 网盘 —— 挂载 WebDAV 服务器，浏览目录并识别影视资源。
 *
 * 三种视图:
 *   sources —— 源列表（添加 / 编辑 / 删除 / 进入）
 *   browse  —— 浏览目录（点目录进入、点视频播放），可一键「识别影视」
 *   scan    —— 识别结果（按片名聚合的电影/剧集），点开选集播放
 *
```

```
 * 播放走 Player 的「直连」链路，通过 PlayParam.header 把 Basic Auth 带过去，
 * 剧集用 Playlist 共享，遥控器可切上/下一集。
 */
import { Feedback } from '../common/Feedback';
import router from '@ohos.router';
import common from '@ohos.app.ability.common';
import { Store } from '../common/Store';
import { PlayParam, WebdavSource, WebdavEntry, WebdavVod, WebdavEp, Episode, WdHis
tory, T } from '../common/Model';
import { Webdav } from '../common/Webdav';
import { Playlist } from '../common/Playlist';
import { Device, KEY_IS_PHONE } from '../common/Device';
@Entry
@Component
struct WebdavPage {
  @State sources: WebdavSource[] = [];
  @State view: string = 'sources';
  @State hoverKey: string = '';
  @State loading: boolean = false;
  @State msg: string = '';
  @State msgColor: string = T.SUB;
  // 当前浏览的源与路径
  @State cur: WebdavSource = new WebdavSource();
  @State curPath: string = '/';
  @State entries: WebdavEntry[] = [];
  // 识别结果
  @State vods: WebdavVod[] = [];
  // 网盘续播记录
  @State wdHist: WdHistory[] = [];
  // 选集弹窗
  @State epOpen: boolean = false;
  @State selVod: WebdavVod = new WebdavVod();
  @StorageProp(KEY_IS_PHONE) isPhone: boolean = false;
  async aboutToAppear(): Promise<void> {
    await Store.init(getContext(this) as common.UIAbilityContext);
    await this.refreshData();
  }
  /** 从播放页/设置页返回时刷新源列表与续播记录 */
  onPageShow(): void {
    this.refreshData();
    // 播放页退出时续播记录是异步落盘的（saveWdPos fire-and-forget），
    // 刚回来那一刻 prefs 可能还没 flush 完，读到旧值 → 「继续观看」不显示。
    // 延迟再读一次，等写入落盘后把记录补出来。
    setTimeout(() => {
      this.refreshData();
    }, 350);
  }
  async refreshData(): Promise<void> {
    this.sources = await Store.webdavSources();
    this.wdHist = await Store.wdHistory();
```

```
}
// ---------- 焦点 ----------
hi(key: string): boolean {
  return this.hoverKey === key;
}
hover(key: string, on: boolean): void {
  if (on) {
    this.hoverKey = key;
  } else if (this.hoverKey === key) {
    this.hoverKey = '';
  }
}
onBackPress(): boolean {
  if (this.epOpen) {
    this.epOpen = false;
    return true;
  }
  if (this.view !== 'sources') {
    this.backToSources();
    return true;
  }
  router.back();
  return true;
}
backToSources(): void {
  this.view = 'sources';
  this.entries = [];
  this.vods = [];
  this.msg = '';
}
// ---------- 浏览 ----------
/**
 * 进入一个源：优先读上次的识别缓存，有就直接展示；没有则自动扫描识别。
 * 目录浏览不再作为默认视图，想看文件树再点「浏览文件」。
 */
async enterSource(src: WebdavSource): Promise<void> {
  this.cur = src;
  this.curPath = Webdav.mountHref(src);
  this.vods = [];
  this.msg = '';
  const cache: WebdavVod[] = await Store.webdavScanCache(src.id);
  if (cache.length > 0) {
    this.vods = cache;
    this.view = 'scan';
    this.msg = '上次识别到 ' + cache.length + ' 部影视，点「重新扫描」可刷新';
    this.msgColor = '#B7F0BC';
    return;
  }
  // 无缓存 → 切到识别视图再自动扫描（扫描中能看到进度）
  this.view = 'scan';
```

```
  await this.scanVods();
}
async loadDir(): Promise<void> {
  this.loading = true;
  this.entries = [];
  try {
    this.entries = await Webdav.list(this.cur, this.curPath);
  } catch (e) {
    const err = e as Error;
    this.msg = err.message;
    this.msgColor = T.WARN;
  }
  this.loading = false;
}
/** 从识别结果视图切回目录浏览 */
async openBrowse(): Promise<void> {
  this.view = 'browse';
  this.msg = '';
  await this.loadDir();
}
async enterDir(entry: WebdavEntry): Promise<void> {
  this.curPath = Webdav.stripSlash(entry.href);
  await this.loadDir();
}
async goUp(): Promise<void> {
  const p: string = Webdav.stripSlash(this.curPath);
  const i: number = p.lastIndexOf('/');
  this.curPath = i <= 0 ? '/' : p.substring(0, i);
  await this.loadDir();
}
pathLabel(): string {
  let p: string = this.curPath;
  try {
    p = decodeURIComponent(p);
  } catch (e) {
    // 保持原文
  }
  return p;
}
// ---------- 播放 ----------
playFile(entry: WebdavEntry): void {
  const v = new WebdavVod();
  v.name = Webdav.stripExt(entry.name);
  v.kind = 'movie';
  v.dir = Webdav.stripSlash(this.curPath);
  const ep = new WebdavEp();
  ep.name = entry.name;
  ep.url = Webdav.absUrl(this.cur, entry.href);
  ep.href = entry.href;
  v.eps.push(ep);
```

```
    this.playEp(v, 0);
  }
  openEp(vod: WebdavVod): void {
    if (vod.eps.length === 1) {
      this.playEp(vod, 0);
      return;
    }
    this.selVod = vod;
    this.epOpen = true;
  }
  epNameOf(vod: WebdavVod, ep: WebdavEp, i: number): string {
    if (vod.kind === 'series') {
      if (ep.season > 0 && ep.episode > 0) {
        return 'S' + WebdavPage.pad2(ep.season) + 'E' + WebdavPage.pad2(ep.episode
);
      }
      if (ep.episode > 0) {
        return '第 ' + ep.episode + ' 集';
      }
    }
    return Webdav.stripExt(ep.name);
  }
  playEp(vod: WebdavVod, idx: number): void {
    if (idx < 0 || idx >= vod.eps.length) {
      return;
    }
    // 交给播放列表，剧集可切上/下一集
    const eps: Episode[] = [];
    for (let i = 0; i < vod.eps.length; i++) {
      const e = new Episode();
      e.name = this.epNameOf(vod, vod.eps[i], i);
      e.url = vod.eps[i].url;
      eps.push(e);
    }
    Playlist.set(eps, idx, vod.name);
    this.epOpen = false;
    const p = new PlayParam();
    p.url = vod.eps[idx].url;
    p.title = vod.name + (vod.eps.length > 1 ? (' ' + eps[idx].name) : '');
    p.siteKey = 'webdav';
    p.vodId = 'webdav:' + this.cur.id + ':' + vod.name;
    p.from = 'direct';
    p.epIndex = idx;
    p.header = Webdav.authHeader(this.cur);
    // 网盘续播落盘所需字段
    p.wdSrcId = this.cur.id;
    p.wdSrcName = this.cur.name;
    p.wdVod = vod.name;
    p.wdDir = vod.dir;
    p.wdSeries = vod.eps.length > 1;
```

```
  router.pushUrl({ url: 'pages/Player', params: p });
}
// ---------- 续播 ----------
/** 从续播记录继续观看：seek 到上次进度，能重建选集时仍可切上/下一集 */
async continueWatch(h: WdHistory): Promise<void> {
  const srcs: WebdavSource[] = await Store.webdavSources();
  let src: WebdavSource | null = null;
  for (let i = 0; i < srcs.length; i++) {
    if (srcs[i].id === h.srcId) {
      src = srcs[i];
      break;
    }
  }
  if (src === null) {
    // 源都删了，这条续播记录必然失效，直接清掉
    await Store.removeWdHistory(h);
    this.wdHist = await Store.wdHistory();
    return;
  }
  // 尝试从识别缓存重建整部剧的选集，续播时也能切上/下一集
  let eps: Episode[] = [];
  let startIdx: number = 0;
  let playUrl: string = h.url;
  let title: string = h.vodName;
  const cache: WebdavVod[] = await Store.webdavScanCache(src.id);
  let vod: WebdavVod | null = null;
  for (let i = 0; i < cache.length; i++) {
    if (cache[i].name === h.vodName) {
      vod = cache[i];
      break;
    }
  }
  if (vod !== null && vod.eps.length > 0) {
    for (let i = 0; i < vod.eps.length; i++) {
      const e = new Episode();
      e.name = this.epNameOf(vod, vod.eps[i], i);
      e.url = vod.eps[i].url;
      eps.push(e);
    }
    startIdx = (h.epIndex >= 0 && h.epIndex < vod.eps.length) ? h.epIndex : 0;
    playUrl = vod.eps[startIdx].url;
    title = vod.name + (vod.eps.length > 1 ? (' ' + eps[startIdx].name) : '');
    Playlist.set(eps, startIdx, vod.name);
  } else if (h.epName.length > 0) {
    title = h.vodName + ' ' + h.epName;
  }
  const p = new PlayParam();
  p.url = playUrl;
  p.title = title;
  p.siteKey = 'webdav';
```

```
    p.vodId = 'webdav:' + src.id + ':' + h.vodName;
    p.from = 'direct';
    p.epIndex = startIdx;
    p.header = Webdav.authHeader(src);
    p.seek = h.pos;
    p.wdSrcId = src.id;
    p.wdSrcName = src.name;
    p.wdVod = h.vodName;
    p.wdDir = h.dir;
    // 是否剧集：能重建选集按集数判断；否则按历史记录粗略判断（epIndex>0 或曾有集
名）
    if (vod !== null && vod.eps.length > 0) {
      p.wdSeries = vod.eps.length > 1;
    } else {
      p.wdSeries = h.epIndex > 0 || h.epName.length > 0;
    }
    router.pushUrl({ url: 'pages/Player', params: p });
  }
  async removeWd(h: WdHistory): Promise<void> {
    await Store.removeWdHistory(h);
    this.wdHist = await Store.wdHistory();
  }
  static pad2(n: number): string {
    return n < 10 ? '0' + n : String(n);
  }
  // ---------- 识别影视 ----------
  async scanVods(): Promise<void> {
    this.loading = true;
    this.msg = '正在扫描目录识别影视资源…';
    this.msgColor = T.SUB;
    try {
      const result: WebdavVod[] = await Webdav.scan(this.cur, 6,
        (dirs: number, files: number): void => {
          this.msg = '正在扫描… 已扫 ' + dirs + ' 个目录，发现 ' + files + ' 个视
频';
        });
      this.vods = result;
      // 识别结果落盘，下次进入直接读缓存
      await Store.saveWebdavScan(this.cur.id, result);
      this.view = 'scan';
      this.msg = '识别到 ' + result.length + ' 部影视';
      this.msgColor = '#B7F0BC';
    } catch (e) {
      const err = e as Error;
      this.msg = err.message;
      this.msgColor = T.WARN;
      // 扫描失败保留在 browse 视图（如果本来就在 browse），否则停在 sources
      if (this.view !== 'browse') {
        this.view = 'browse';
        await this.loadDir();
```

```
      }
    }
    this.loading = false;
  }
  // ---------- UI ----------
  @Builder
  Btn(label: string, key: string, primary: boolean, act: () => void) {
    Text(label)
      .fontSize(15)
      .height(44)
      .padding({ left: 22, right: 22 })
      .borderRadius(T.RADIUS_PILL)
      .backgroundColor(primary ? T.ACCENT : (this.hi(key) ? T.CARD_HI : T.CARD))
      .fontColor(primary ? T.BG : (this.hi(key) ? T.SELECTED : T.TXT))
      .border({ width: T.BORDER, color: this.hi(key) ? (primary ? T.SELECTED : T.T
XT) : '#00000000' })
      .textAlign(TextAlign.Center)
      .animation({ duration: T.ANIM, curve: Curve.EaseOut })
      .focusable(true)
      .onFocus(() => this.hover(key, true))
      .onBlur(() => this.hover(key, false))
      .onHover((hv: boolean) => this.hover(key, hv))
      .onClick(() => {
        Feedback.tap();
        act();
      })
  }
  /** 通用行底板：标题 + 说明 + 右侧操作文字，焦点高亮 */
  @Builder
  RowItem(key: string, title: string, sub: string, right: string, act: () => void)
 {
    Row() {
      Column() {
        Text(title).fontSize(15).fontColor(T.TXT).maxLines(1).textOverflow({ overf
low: TextOverflow.Ellipsis })
        if (sub.length > 0) {
          Text(sub).fontSize(12).fontColor(T.DIM).margin({ top: 3 }).maxLines(1)
            .textOverflow({ overflow: TextOverflow.Ellipsis })
        }
      }
      .alignItems(HorizontalAlign.Start)
      .layoutWeight(1)
      if (right.length > 0) {
        Text(right).fontSize(13).fontColor(T.SELECTED).margin({ left: 10 })
      }
    }
    .width('100%')
    .height(56)
    .padding({ left: 14, right: 14 })
    .borderRadius(T.RADIUS_SM)
```

```
  .backgroundColor(this.hi(key) ? T.CARD_HI : T.CARD)
  .border({ width: T.BORDER, color: this.hi(key) ? T.TXT : '#00000000' })
  .margin({ bottom: 8 })
  .animation({ duration: T.ANIM, curve: Curve.EaseOut })
  .focusable(true)
  .onFocus(() => this.hover(key, true))
  .onBlur(() => this.hover(key, false))
  .onHover((hv: boolean) => this.hover(key, hv))
  .onClick(() => {
    Feedback.tap();
    act();
  })
}
@Builder
TopBar(title: string) {
  Row() {
    Text('‹')
      .fontSize(34)
      .fontColor(this.hi('back') ? T.TXT : T.SUB)
      .width(40)
      .textAlign(TextAlign.Center)
      .focusable(true)
      .defaultFocus(true)
      .onFocus(() => this.hover('back', true))
      .onBlur(() => this.hover('back', false))
      .onHover((hv: boolean) => this.hover('back', hv))
      .onClick(() => {
        Feedback.tap();
        if (this.view === 'sources') {
          router.back();
        } else {
          this.backToSources();
        }
      })
    Text(title).fontSize(24).fontColor(T.TXT).margin({ left: 8 })
    if (this.view === 'browse') {
      Text(this.cur.name + '  ·  ' + this.pathLabel())
        .fontSize(14).fontColor(T.DIM).margin({ left: 16 }).maxLines(1)
        .textOverflow({ overflow: TextOverflow.Ellipsis }).layoutWeight(1)
    } else if (this.view === 'scan') {
      Text(this.cur.name)
        .fontSize(14).fontColor(T.DIM).margin({ left: 16 }).maxLines(1)
        .textOverflow({ overflow: TextOverflow.Ellipsis }).layoutWeight(1)
    }
  }
  .width('100%')
  .padding({ left: 24, right: 24, top: 24, bottom: 8 })
}
build() {
  Stack({ alignContent: Alignment.Center }) {
```

```
      Column() {
        this.TopBar(this.view === 'sources' ? 'WebDAV 网盘' : (this.view === 'brow
se' ? '浏览文件' : '识别影视'))
        Scroll() {
          Column() {
            if (this.view === 'sources') {
              this.sourcesView()
            } else if (this.view === 'browse') {
              this.browseView()
            } else {
              this.scanView()
            }
          }
          .width('100%')
          .padding({ left: 24, right: 24, bottom: 30 })
          .alignItems(HorizontalAlign.Start)
        }
        .layoutWeight(1)
        .width('100%')
        .scrollBar(BarState.Off)
      }
      .width('100%')
      .height('100%')
      .backgroundColor(T.BG)
      if (this.epOpen) {
        this.epPanel()
      }
    }
    .width('100%')
    .height('100%')
  }
  @Builder
  sourcesView() {
    Column() {
      // 继续观看：网盘播放记录，点击从上次进度续播
      if (this.wdHist.length > 0) {
        Row() {
          Text('继续观看').fontSize(17).fontColor(T.TXT)
          Blank()
          Text('清空').fontSize(13).fontColor(T.DIM)
            .padding({ left: 12, right: 12, top: 6, bottom: 6 })
            .borderRadius(T.RADIUS_PILL)
            .backgroundColor(this.hi('clearWd') ? T.CARD_HI : T.CARD)
            .border({ width: T.BORDER, color: this.hi('clearWd') ? T.TXT : '#00000
000' })
            .focusable(true)
            .onFocus(() => this.hover('clearWd', true))
            .onBlur(() => this.hover('clearWd', false))
            .onHover((hv: boolean) => this.hover('clearWd', hv))
            .onClick(() => {
```

```
            Feedback.tap();
            this.clearWd();
          })
        }
        .width('100%')
        .margin({ top: 6, bottom: 8 })
        ForEach(this.wdHist, (h: WdHistory) => {
          this.wdHistItem(h)
        }, (h: WdHistory) => h.srcId + '_' + h.vodName)
        Text('挂载的网盘在这里浏览目录、识别并播放影视资源；新增 / 编辑 / 删除网盘
请到「设置 → 网盘」。')
          .fontSize(14).fontColor(T.SUB).width('100%').lineHeight(22).margin({ top
: 18 })
      } else {
        Text('挂载的网盘在这里浏览目录、识别并播放影视资源；新增 / 编辑 / 删除网盘
请到「设置 → 网盘」。')
          .fontSize(14).fontColor(T.SUB).width('100%').lineHeight(22)
      }
      Row() {
        this.Btn('去设置页添加', 'go_setting', true, () => {
          router.pushUrl({ url: 'pages/SettingPage' });
        })
        Blank()
      }
      .width('100%')
      .margin({ top: 16, bottom: 6 })
      if (this.sources.length === 0) {
        Text('还没有挂载任何网盘。点上方「去设置页添加」，填服务器地址、账号密码即
可。')
          .fontSize(14).fontColor(T.DIM).width('100%').margin({ top: 20 }).lineHei
ght(22)
      } else {
        ForEach(this.sources, (s: WebdavSource) => {
          this.RowItem('s_' + s.id, s.name, s.url, '进入 ›', () => {
            this.enterSource(s);
          })
        }, (s: WebdavSource) => s.id)
      }
    }
  }
  /** 单条续播记录 */
  @Builder
  wdHistItem(h: WdHistory) {
    Row() {
      Column() {
        Text(h.vodName).fontSize(15).fontColor(T.TXT).maxLines(1)
          .textOverflow({ overflow: TextOverflow.Ellipsis })
        Row() {
          Text(h.epName.length > 0 ? ('看到 ' + h.epName) : '电影')
            .fontSize(12).fontColor(T.DIM)
```

```
          if (h.pos > 0) {
            Text(' · 已看 ' + WebdavPage.fmtPos(h.pos) + (h.dur > 0 ? (' / ' + Web
davPage.fmtPos(h.dur)) : ''))
              .fontSize(12).fontColor(T.SELECTED)
          }
        }
        .margin({ top: 3 })
      }
      .alignItems(HorizontalAlign.Start)
      .layoutWeight(1)
      .onClick(() => {
        Feedback.tap();
        this.continueWatch(h);
      })
      Text('继续播放 ›')
        .fontSize(13).fontColor(T.SELECTED).margin({ left: 10 })
        .onClick(() => {
          Feedback.tap();
          this.continueWatch(h);
        })
      Text('×')
        .fontSize(15).fontColor(T.DIM)
        .width(36).height(36).textAlign(TextAlign.Center).borderRadius(18)
        .margin({ left: 8 })
        .onClick(() => {
          Feedback.tap();
          this.removeWd(h);
        })
    }
    .width('100%')
    .height(56)
    .padding({ left: 14, right: 14 })
    .borderRadius(T.RADIUS_SM)
    .backgroundColor(this.hi('w_' + h.srcId + '_' + h.vodName) ? T.CARD_HI : T.CAR
D)
    .border({ width: T.BORDER, color: this.hi('w_' + h.srcId + '_' + h.vodName) ?
T.TXT : '#00000000' })
    .margin({ bottom: 8 })
    .animation({ duration: T.ANIM, curve: Curve.EaseOut })
    .focusable(true)
    .onFocus(() => this.hover('w_' + h.srcId + '_' + h.vodName, true))
    .onBlur(() => this.hover('w_' + h.srcId + '_' + h.vodName, false))
    .onHover((hv: boolean) => this.hover('w_' + h.srcId + '_' + h.vodName, hv))
  }
  async clearWd(): Promise<void> {
    await Store.clearWdHistory();
    this.wdHist = [];
  }
  @Builder
  browseView() {
```

```
    Column() {
      Row() {
        this.Btn('识别影视', 'scan', true, () => {
          this.scanVods();
        })
        this.Btn('上级目录', 'up', false, () => {
          this.goUp();
        })
        this.Btn('刷新', 'refresh', false, () => {
          this.loadDir();
        })
        Blank()
      }
      .width('100%')
      .margin({ top: 8, bottom: 12 })
      if (this.loading) {
        Row() {
          LoadingProgress().width(24).height(24).color(T.SUB)
          Text('加载中…').fontSize(14).fontColor(T.DIM).margin({ left: 10 })
        }
        .width('100%').margin({ top: 20 })
      } else if (this.entries.length === 0) {
        Text(this.msg.length > 0 ? this.msg : '这个目录是空的。')
          .fontSize(14).fontColor(this.msgColor).width('100%').margin({ top: 16 })
.lineHeight(22)
      } else {
        ForEach(this.entries, (e: WebdavEntry, i: number) => {
          this.RowItem(
            'f_' + i,
            (e.isDir ? '📁 ' : '🎬 ') + e.name,
            e.isDir ? '目录' : (WebdavPage.sizeLabel(e.size) + (e.mtime.length > 0
 ? ' · ' + e.mtime : '')),
            e.isDir ? '进入' : '播放',
            () => {
              if (e.isDir) {
                this.enterDir(e);
              } else if (Webdav.isVideo(e.name)) {
                this.playFile(e);
              }
            })
        }, (e: WebdavEntry, i: number) => e.href + i)
      }
    }
  }
  @Builder
  scanView() {
    Column() {
      Row() {
        this.Btn('浏览文件', 'browse', false, () => {
          this.openBrowse();
```

```
        })
        this.Btn('重新扫描', 'rescan', true, () => {
          this.scanVods();
        })
        Blank()
      }
      .width('100%')
      .margin({ top: 8, bottom: 12 })
      if (this.loading) {
        Column() {
          Row() {
            LoadingProgress().width(24).height(24).color(T.SUB)
            Text('扫描中…').fontSize(14).fontColor(T.DIM).margin({ left: 10 })
          }
          if (this.msg.length > 0) {
            Text(this.msg).fontSize(13).fontColor(T.DIM).width('100%').margin({ to
p: 10 }).lineHeight(20)
          }
        }
        .width('100%').margin({ top: 20 })
      } else if (this.vods.length === 0) {
        Text(this.msg.length > 0 ? this.msg : '没有识别到视频资源。')
          .fontSize(14).fontColor(this.msgColor).width('100%').margin({ top: 16 })
.lineHeight(22)
      } else {
        ForEach(this.vods, (v: WebdavVod, i: number) => {
          this.RowItem('v_' + i, v.name, this.vodMeta(v), '播放', () => {
            this.openEp(v);
          })
        }, (v: WebdavVod, i: number) => v.name + i)
      }
    }
  }
  @Builder
  epPanel() {
    Stack() {
      Column()
        .width('100%').height('100%')
        .backgroundColor(T.BLACK_70)
        .onClick(() => {
          Feedback.tap();
          this.epOpen = false;
        })
      Column() {
        Row() {
          Text(this.selVod.name).fontSize(19).fontColor(T.TXT).maxLines(1)
            .textOverflow({ overflow: TextOverflow.Ellipsis }).layoutWeight(1)
          Blank()
          Text('×').fontSize(19).fontColor(this.hi('ex') ? T.TXT : T.SUB)
            .width(42).height(42).textAlign(TextAlign.Center).borderRadius(T.RADIU
```

```
S_SM)
            .backgroundColor(this.hi('ex') ? T.CARD_HI : '#00000000')
            .focusable(true).defaultFocus(true)
            .onFocus(() => this.hover('ex', true))
            .onBlur(() => this.hover('ex', false))
            .onHover((hv: boolean) => this.hover('ex', hv))
            .onClick(() => {
              Feedback.tap();
              this.epOpen = false;
            })
        }
        .width('100%')
        Scroll() {
          Column() {
            ForEach(this.selVod.eps, (ep: WebdavEp, i: number) => {
              this.RowItem('ep_' + i, this.epNameOf(this.selVod, ep, i), '', '播放
', () => {
                this.playEp(this.selVod, i);
              })
            }, (ep: WebdavEp, i: number) => ep.href + i)
          }
          .width('100%')
        }
        .width('100%')
        .height(this.isPhone ? 320 : 360)
        .margin({ top: 14 })
        .scrollBar(BarState.Off)
      }
      .width(Device.panelW(520))
      .padding(24)
      .backgroundColor(T.SURFACE)
      .borderRadius(28)
      .alignItems(HorizontalAlign.Start)
      .focusScopeId('ep_panel', true)
    }
    .width('100%')
    .height('100%')
  }
  vodMeta(v: WebdavVod): string {
    return (v.kind === 'series' ? '剧集 · ' : '电影 · ') +
      (v.year.length > 0 ? v.year + ' · ' : '') + v.eps.length + ' 集';
  }
  static sizeLabel(size: number): string {
    if (size <= 0) {
      return '';
    }
    if (size >= 1024 * 1024 * 1024) {
      return (size / 1024 / 1024 / 1024).toFixed(2) + ' GB';
    }
    if (size >= 1024 * 1024) {
```

```
      return (size / 1024 / 1024).toFixed(1) + ' MB';
    }
    if (size >= 1024) {
      return (size / 1024).toFixed(0) + ' KB';
    }
    return size + ' B';
  }
  static fmtPos(ms: number): string {
    if (ms <= 0) {
      return '0:00';
    }
    const t: number = Math.floor(ms / 1000);
    const h: number = Math.floor(t / 3600);
    const m: number = Math.floor((t % 3600) / 60);
    const s: number = t % 60;
    const mm: string = (m < 10 ? '0' : '') + m;
    const ss: string = (s < 10 ? '0' : '') + s;
    return h > 0 ? (h + ':' + mm + ':' + ss) : (mm + ':' + ss);
  }
}
// ======================================================================
// 文件：entry/src/main/ets/common/Quality.ets     （共 368 行）
// ======================================================================
/**
 * 清晰度与网络自适应。
 *
 * 卡顿的根子通常是「码率 > 实际带宽」。能做的三件事：
 *  1. 认出 HLS master playlist 里的多档码率，让用户能选，也能自动选；
 *  2. 用 AVPlayer 的 PlaybackStrategy.preferredHeight 把档位钉在选定值上
 *     （保留 master 地址，系统的 ABR 仍然能工作，比我们自己换流地址稳）；
 *  3. 缓冲水位 preferredBufferDuration 按实测网速抬高，慢网多垫几秒就不容易断。
 */
import http from '@ohos.net.http';
const UA: string = 'Mozilla/5.0 (Linux; HarmonyOS; SmartScreen) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36';
/** 一档清晰度 */
export class QLevel {
  /** 竖向分辨率，0 表示源没标 RESOLUTION */
  height: number = 0;
  width: number = 0;
  /** 声明峰值码率，bps */
  bps: number = 0;
  /** 这一档的媒体播放列表地址 */
  url: string = '';
  /** 源里自带的名字，如 "高清"、"1080p" */
  name: string = '';
}
/** 一次测速结果 */
export class QSpeed {
  /** 实测下行速率 kbps，-1 表示没测出来 */
```

```
  kbps: number = -1;
  bytes: number = 0;
  ms: number = 0;
}
export class Quality {
  /** 「自动」档 */
  static readonly AUTO: number = 0;
  /** 「原画」= 不设上限，取源里最高 */
  static readonly TOP: number = 9999;
  /**
   * 解析 master playlist。
   * 形如：
   *   #EXT-X-STREAM-INF:BANDWIDTH=2800000,RESOLUTION=1920x1080,NAME="高清"
   *   1080p/index.m3u8
   * 只有带 EXT-X-STREAM-INF 的才算多档；普通媒体列表返回空数组。
   */
  static parseMaster(text: string, base: string): QLevel[] {
    const out: QLevel[] = [];
    if (text.indexOf('#EXT-X-STREAM-INF') < 0) {
      return out;
    }
    const lines: string[] = text.replace(/\r\n/g, '\n').split('\n');
    let pending: QLevel | null = null;
    for (let i = 0; i < lines.length; i++) {
      const line: string = lines[i].trim();
      if (line.length === 0) {
        continue;
      }
      if (line.startsWith('#EXT-X-STREAM-INF')) {
        const lv = new QLevel();
        lv.bps = Quality.numAttr(line, 'AVERAGE-BANDWIDTH');
        const peak: number = Quality.numAttr(line, 'BANDWIDTH');
        if (peak > 0) {
          lv.bps = peak;
        }
        const res: string = Quality.strAttr(line, 'RESOLUTION');
        if (res.length > 0) {
          const x: number = res.toLowerCase().indexOf('x');
          if (x > 0) {
            lv.width = Quality.toNum(res.substring(0, x));
            lv.height = Quality.toNum(res.substring(x + 1));
          }
        }
        lv.name = Quality.strAttr(line, 'NAME');
        if (lv.name.length === 0) {
          lv.name = Quality.strAttr(line, 'VIDEO-RANGE');
        }
        pending = lv;
        continue;
      }
```

```
    if (line.startsWith('#')) {
      continue;
    }
    if (pending !== null) {
      pending.url = Quality.abs(base, line);
      out.push(pending);
      pending = null;
    }
  }
  // 按清晰度升序，没有 RESOLUTION 的按码率排
  out.sort((a: QLevel, b: QLevel): number => {
    const ha: number = a.height > 0 ? a.height : Math.floor(a.bps / 20000);
    const hb: number = b.height > 0 ? b.height : Math.floor(b.bps / 20000);
    if (ha !== hb) {
      return ha - hb;
    }
    return a.bps - b.bps;
  });
  return out;
}
/** 显示名。优先用分辨率，没标就用码率折算 */
static labelOf(h: number, bps: number): string {
  if (h >= 2160) {
    return '4K';
  }
  if (h >= 1440) {
    return '2K';
  }
  if (h >= 1080) {
    return '1080P';
  }
  if (h >= 720) {
    return '720P';
  }
  if (h >= 540) {
    return '540P';
  }
  if (h >= 480) {
    return '480P';
  }
  if (h >= 360) {
    return '360P';
  }
  if (bps >= 4000000) {
    return '高清';
  }
  if (bps >= 1500000) {
    return '标清';
  }
  return bps > 0 ? '流畅' : '原画';
```

```
  }
  static labelOfLevel(l: QLevel): string {
    const s: string = Quality.labelOf(l.height, l.bps);
    if (s === '原画' && l.name.length > 0) {
      return l.name;
    }
    // labelOf 只认 360/480/540/720/1080 这些标准高度，其余一律落到
    // 「流畅/标清/高清」三档，于是 240P 和 180P 会显示成两个一模一样的「流畅」，
    // 面板上根本分不清。这类档位直接显示真实高度。
    const vague: boolean = s === '流畅' || s === '标清' || s === '高清';
    if (vague && l.height > 0) {
      return l.height + 'P';
    }
    // 连高度都没写的 master 只能靠码率区分
    if (vague && l.bps > 0) {
      return s + ' ' + Math.round(l.bps / 1000) + 'K';
    }
    return s;
  }
  /**
   * 按实测速率挑档：留 30% 余量，取「码率不超过 0.7×速率」的最高一档。
   * 一档都不满足（网太差）就退到最低档，至少别整个卡死。
   */
  static pickBySpeed(kbps: number, levels: QLevel[]): number {
    if (levels.length === 0) {
      return -1;
    }
    if (kbps <= 0) {
      return 0;
    }
    const budget: number = kbps * 0.7 * 1000;
    let best: number = -1;
    for (let i = 0; i < levels.length; i++) {
      const need: number = levels[i].bps > 0 ? levels[i].bps : (levels[i].height >
0 ? levels[i].height * 4000 : 0);
      if (need <= budget) {
        best = i;
      }
    }
    return best < 0 ? 0 : best;
  }
  /** 选「不高于 cap 的最高档」；cap 为 TOP 时直接取最高档 */
  static pickByCap(cap: number, levels: QLevel[]): number {
    if (levels.length === 0) {
      return -1;
    }
    if (cap >= Quality.TOP) {
      return levels.length - 1;
    }
    let best: number = -1;
```

```
  for (let i = 0; i < levels.length; i++) {
    if (levels[i].height <= 0 || levels[i].height <= cap) {
      best = i;
    }
  }
  return best < 0 ? 0 : best;
}
/** 从媒体列表里找第一个分片，用来实测网速 */
static firstSegment(playlist: string, base: string): string {
  const lines: string[] = playlist.replace(/\r\n/g, '\n').split('\n');
  let nextIsSeg: boolean = false;
  for (let i = 0; i < lines.length; i++) {
    const line: string = lines[i].trim();
    if (line.length === 0) {
      continue;
    }
    if (line.startsWith('#EXTINF')) {
      nextIsSeg = true;
      continue;
    }
    if (line.startsWith('#')) {
      continue;
    }
    if (nextIsSeg || line.indexOf('.ts') > 0 || line.indexOf('.m4s') > 0 ||
      line.indexOf('.mp4') > 0 || line.indexOf('.flv') > 0) {
      return Quality.abs(base, line);
    }
  }
  return '';
}
/**
 * 实测下行速率。
 *
 * 用 dataReceive 边收边计数，收到 limit 就主动 destroy——
 * 不这样做的话，遇到不支持 Range 的 CDN 会把整个 mp4 拉下来，几百兆。
 */
static measure(url: string, ms: number, limit: number): Promise<QSpeed> {
  return new Promise<QSpeed>((resolve) => {
    const st = new QSpeed();
    if (url.length === 0) {
      resolve(st);
      return;
    }
    const req = http.createHttp();
    let bytes: number = 0;
    let settled: boolean = false;
    let timer: number = -1;
    const t0: number = Date.now();
    const finish = (): void => {
      if (settled) {
```

```
        return;
      }
      settled = true;
      if (timer >= 0) {
        clearTimeout(timer);
        timer = -1;
      }
      try {
        req.off('dataReceive');
      } catch (e) {
        // 忽略
      }
      try {
        req.destroy();
      } catch (e) {
        // 忽略
      }
      st.bytes = bytes;
      st.ms = Date.now() - t0;
      // bits/ms 就是 kbps：1 字节/毫秒 = 8 kbps
      if (st.ms > 150 && bytes > 16384) {
        st.kbps = Math.floor(bytes * 8 / st.ms);
      }
      resolve(st);
    };
    try {
      req.on('dataReceive', (buf: ArrayBuffer) => {
        bytes += buf.byteLength;
        if (bytes >= limit) {
          finish();
        }
      });
    } catch (e) {
      // 老版本没有 dataReceive，退化成「只测得没测速」
    }
    timer = setTimeout(finish, ms);
    const opt: http.HttpRequestOptions = {
      method: http.RequestMethod.GET,
      header: { 'User-Agent': UA, 'Accept': '*/*', 'Range': 'bytes=0-' + (limit
- 1) },
      connectTimeout: ms,
      readTimeout: ms,
      usingCache: false
    };
    req.request(url, opt).then(() => {
      finish();
    }).catch(() => {
      // 连接失败/被 destroy：有多少算多少，够 16KB 就给结果
      finish();
    });
```

```
  });
}
static fmtKbps(kbps: number): string {
  if (kbps <= 0) {
    return '';
  }
  if (kbps >= 1000) {
    return (Math.round(kbps / 100) / 10) + ' MB/s';
  }
  return kbps + ' KB/s';
}
// ---------- 内部 ----------
private static toNum(s: string): number {
  const n: number = parseInt(s.trim(), 10);
  return isNaN(n) ? 0 : n;
}
private static numAttr(line: string, key: string): number {
  return Quality.toNum(Quality.strAttr(line, key));
}
private static strAttr(line: string, key: string): string {
  const i: number = line.indexOf(key + '=');
  if (i < 0) {
    return '';
  }
  let s: number = i + key.length + 1;
  let e: number = line.length;
  if (line.charAt(s) === '"') {
    s = s + 1;
    const q: number = line.indexOf('"', s);
    return q < 0 ? line.substring(s) : line.substring(s, q);
  }
  const comma: number = line.indexOf(',', s);
  if (comma >= 0) {
    e = comma;
  }
  return line.substring(s, e).trim();
}
/** 相对地址转绝对（HLS 里分片地址常见 "../xxx/seg.ts"） */
static abs(base: string, ref: string): string {
  const r: string = ref.trim();
  if (r.startsWith('http://') || r.startsWith('https://')) {
    return r;
  }
  const scheme: number = base.indexOf('://');
  if (scheme < 0) {
    return r;
  }
  const hostEnd: number = base.indexOf('/', scheme + 3);
  const origin: string = hostEnd < 0 ? base : base.substring(0, hostEnd);
  if (r.startsWith('//')) {
```

```
      return base.substring(0, scheme) + ':' + r;
    }
    if (r.startsWith('/')) {
      return origin + r;
    }
    const cut: number = base.indexOf('?');
    const clean: string = cut >= 0 ? base.substring(0, cut) : base;
    const dir: string = clean.substring(0, clean.lastIndexOf('/') + 1);
    const path: string = dir + r;
    // 只对 origin 之后的路径段做 ../ 归一，否则会把 http:// 里的斜杠吃掉
    const rest: string = path.length > origin.length ? path.substring(origin.lengt
h) : path;
    const parts: string[] = rest.split('/');
    const stack: string[] = [];
    for (let i = 0; i < parts.length; i++) {
      const p: string = parts[i];
      if (p === '..') {
        if (stack.length > 0) {
          stack.pop();
        }
      } else if (p !== '.' && p.length > 0) {
        stack.push(p);
      }
    }
    return origin + '/' + stack.join('/');
  }
}
// ============================================================================
// 源程序结束
// 软件名称：夜帧影视软件    版本号：V1.0
// 开发完成日期：2026 年 11 月 04 日
// ============================================================================
//
//
//
```